OpenGeoSys Training KIGAM / SNU 2026

Korean Institute for Geosciences and Mineral Resources
Seoul National University

Lecture 1: Course Overview and Introduction to OpenGeoSys

Lecture 2: Continuum Mechanics

Lecture 3: Hydromechanics and Groundwater Flow

Exercise 1: Theis: Wellbore Hydraulics

Exercise 2: STIMTEC: Hydraulic Injection Tests - Single Fracture Model

Exercise 3: STIMTEC: Hydraulic Injection Tests - Discrete Fracture Network Model

Exercise 4: Selke: Catchment Model

Lecture 4: Digital Twins of Geosystems

Olaf Kolditz
Helmholtz Centre for Environmental Research – UFZ, Leipzig
and Dresden University of Technology

Mehran Ghasabeh, Mostafa Mollaali
Technische Universität Bergakademie Freiberg – TUABF

Haibing Shao, Linus Walter, Niklas Ritter
Helmholtz Centre for Environmental Research – UFZ, Leipzig

Kristhian Leandro Peña Cerón, Jörg Renner
Ruhr-Universität Bochum RUB

October 11, 2026

This script, which is based on a systematic OGS study of H processes, was supported by AI (Claude-Opus 5.5)

Contents

1 Lecture 1: Course Overview and Introduction to OpenGeoSys **5**

2 Lecture 2: Continuum Mechanics **12**

3 Lecture 3: Hydromechanics and Groundwater Flow **18**

4 Exercise 1: Theis: Wellbore Hydraulics **25**

5 Exercise 2: STIMTEC: Hydraulic Injection Tests - Single Fracture Model **33**

6 Exercise 3: STIMTEC: Hydraulic Injection Tests - Discrete Fracture Network Model **47**

7 Exercise 4: Selke: Catchment Model **60**

8 Lecture 4: Digital Twins of Geosystems **66**

Course summary

This document describes the training and development of OpenGeoSys (OGS), an open-source software for simulating thermo-hydro-mechanical-chemical (THMC) processes in porous and fractured media.

Overview of the Course Structure

- The course rests on three pillars: lectures, exercises, and the OGS framework.
- There are two thematic strands, hydromechanics and geomechanics, with increasing complexity.

History and Development of OpenGeoSys

- Over 40 years, OGS evolved from the Fortran code RockFlow into a modern C++ framework.
- The focus has been on applications in geothermal energy, water resources management, geotechnics, and nuclear waste disposal.

Software Ecosystem and Relevance

- The ecosystem consists of the core module ogs^{THMC}, surrounded by Python-based tools such as $\text{ogs}^{\text{ogsTools}}$.
- It supports connectivity to external codes, data, GIS, and AI, and promotes reproducibility.

Principles of Scientific Work at OGS

- Focus on reproducible workflows, version control, and open resources.
- Continuous integration and benchmarking ensure the validity of the models.

Fundamental Principles of Continuum Mechanics

- The conservation laws for mass, momentum, and energy in porous media are used.
- The governing equations are determined by the choice of the conserved quantity and the corresponding fluxes.

Differentiation of Fluxes

- The total flux consists of an advective part (with the average velocity) and a diffusive part (Fick's law).
- Fick's law describes diffusive transport processes by means of a diffusion coefficient.

Application to Groundwater and Porous Media

- Mass conservation leads to the continuity equation for water pressure.
- Simplified models, such as divergence-free flow, are used to obtain analytical solutions.

Governing Equation of Groundwater Flow

- Flow in a porous medium is formulated as a diffusion equation in terms of the hydraulic head.
- The balance is either stationary (Laplace equation) or transient (parabolic), with diffusivity K/S_0.

Mathematical Formulation with Hydraulic Head

- The principle separates the properties of the rock (K, S_0) from those of the fluid (ρ, μ).
- The complete formulation enables coupled simulations; in the steady-state case, the solution takes the form of a potential (harmonic) function.

Methodology of Digital Twin Development

- Digital twins are based on reproducible, automated workflows, not merely on models.
- Continuous integration of data and models is central to real-time monitoring and prediction.

Software and Workflow Infrastructure

- Software engineering practices such as version control, automated testing, and container technology ensure reproducibility.
- Containers bundle applications together with their dependencies and facilitate cross-platform use.

Geoscientific Modelling and Scales

- The choice of model depends on complexity and scale, e.g. TH2M in the laboratory or THM in repositories.
- Benchmarking guarantees reliability, with documented validations for all processes.

Applications and Scenarios

- Models for simulating large regions, underground research laboratories, or repositories are linked together.
- Scenarios such as seasonal changes or changes in water level test the robustness of the models.

Digital Twin of the Subsurface

- A digital twin connects geology, sensor data, and models in order to represent relevant properties and behaviour.
- The goal is a continuous, automated feedback loop between data and models for monitoring and control.

Significance and Challenges

- Digital twins are less stand-alone models than ongoing workflows.
- Assurance through verification, validation, and uncertainty quantification is essential.

1 Lecture 1: Course Overview and Introduction to OpenGeoSys

OpenGeoSys Training Korea/China 2026

Lecture 0: Course Overview and Introduction to OpenGeoSys

Olaf Kolditz
Helmholtz Centre for Environmental Research – UFZ, Leipzig
Mehran Ghasabeh
Technische Universität Bergakademie Freiberg – TUBAF

7 October 2026

Abstract

The introductory lecture of the OpenGeoSys (OGS) training course 2026 in Daejeon/Hangzhou (Olaf Kolditz, Mehran Ghasabeh) has two purposes: it sets out the structure of the course, and it introduces OpenGeoSys as a scientific open-source framework for thermo-hydro-mechanical-chemical (THMC) processes in porous and fractured media. The course combines three lecture blocks (continuum mechanics, hydromechanics, geomechanics) with exercises that move from software installation to three hydrological applications (Theis pumping test, STIMTEC hydraulic tests, Selke catchment) and three mechanical and coupled applications (Kirsch excavation, fault reactivation, a THM field-scale experiment). The OGS part of the lecture follows the code over four decades, from the Fortran code RockFlow (1986) to the current C++ release series, and presents the elements that today make OGS more than a solver: a continuous-integration infrastructure, the Python package OGSTools, the Benchmark Gallery and the ModelHub. This essay summarises the lecture and argues that its central message is reproducibility: a simulation result is only as useful as the workflow that allows others to repeat, check and extend it.

1 Introduction

An introductory lecture has to do more than list topics. It has to give students a map of the course and a reason to care about the tool they are going to use for the following days. Lecture 0 of the OpenGeoSys training course, held in Daejeon/Hangzhou on 7 October 2026 for participants from Korea and China, does this in two steps. The first part, the *course overview*, describes the "compass" of the course: which lectures are given, which exercises belong to them, and how the material is distributed (Overleaf for the lecture notes, GitHub for the exercises). The second part, the *introduction to* OpenGeoSys, explains what OpenGeoSys is, where it comes from, how it is developed, and which resources a new user can rely on. The lecture closes with a first practical step: the installation of OGS.

This essay follows the same structure. Section 2 describes the organisation of the course, Sect. 3 the scope and history of OpenGeoSys, Sect. 4 the software ecosystem around the simulator, and Sect. 5 discusses what the lecture implies for how the course should be used.

2 Course structure

The course rests on three pillars (Fig. 1): *lectures* on the physical and mathematical background, *exercises* with OpenGeoSys, and the *OGS framework* itself, which is presented as a family of tools (`ogsTHMC`, `ogsTools`, `ogsWorkflows`, `ogsDigitalTwins`) rather than as a single program. The THMC tetrahedron, with thermal (T), hydraulic (H), mechanical (M) and chemical (C)

processes at its corners, serves as the visual key to the whole course: each exercise addresses one corner or one edge of it.

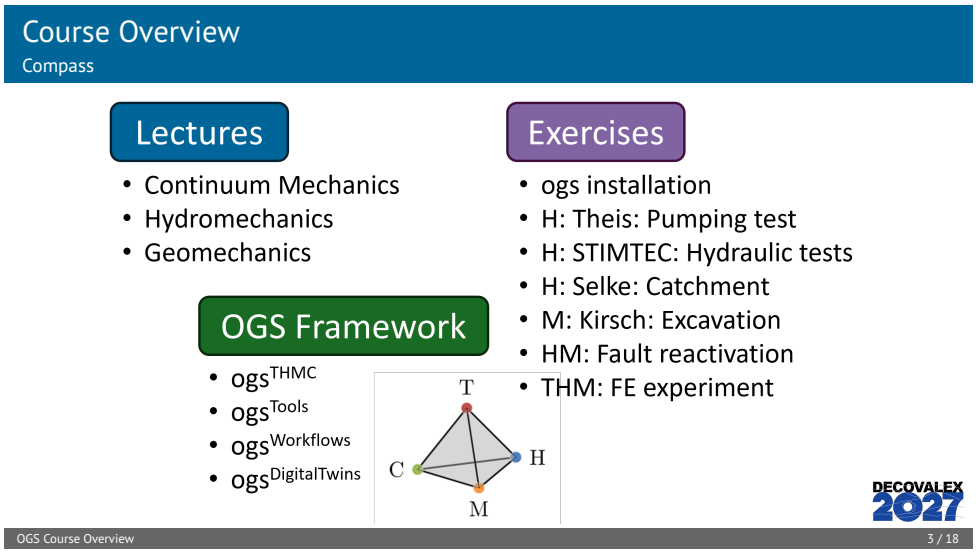


Figure 1: Course "compass": lectures, exercises and the OGS framework, organised by the THMC tetrahedron (slide 3 of the lecture).

The course is split into two strands. The *hydromechanics* strand (Olaf Kolditz) consists of four lectures and four exercises (Table 1). Its exercises increase in complexity: a homogeneous aquifer with an analytical reference solution (Theis), a field test in fractured crystalline rock that requires calibration against measurements (STIMTEC), and a catchment model at the regional scale (Selke). The *geomechanics* strand (Mehran Ghasabeh) adds the mechanical and coupled exercises: the Kirsch problem of stresses around an excavation, the reactivation of a fault under hydromechanical loading, and a thermo-hydro-mechanical field-scale experiment. Its material is provided in a separate GitHub repository (`https://github.com/mehranqsb/OGSTrainingCourse_KIGAM_ZJU`).

Table 1: Lectures and exercises of the hydromechanics strand.

Lectures		Exercises	
L0	Course overview | OpenGeoSys introduction	E0	OGS basics, Benchmark Gallery, GitHub
L1	Continuum mechanics	E1	Theis: pumping well
L2	Hydromechanics	E2	STIMTEC: pumping test in fractured rock
L3	Groundwater modelling	E3	Selke: catchment modelling

The course material is open from the start: lecture notes are written collaboratively on Overleaf and exercises are hosted on GitHub (`https://github.com/OlafKolditz/ogs-course-2026`). This is not only a matter of convenience. It anticipates the main theme of the OGS introduction, namely that scientific software and the models built with it should be versioned, shared and reproducible.

3 OpenGeoSys: scope and history

3.1 THMC processes and applications

OpenGeoSys is a scientific open-source code for the simulation of coupled THMC processes in porous and fractured media. The lecture links the four process classes to typical fields of

application: geothermal energy (T), water resources and geohydrology (H), geotechnical systems and geomechanics (M), and water quality and reactive transport (C). The strength of the framework lies in the coupling of these processes, for example hydromechanics in fractured rock (the STIMTEC exercise) or thermo-hydro-mechanics in the near field of a nuclear waste repository.

3.2 Four decades of development

The history of OpenGeoSys (Fig. 2) is also a history of scientific software engineering. Development began in 1986 with RockFlow (RF1), written in Fortran. RF2 (1989) was the common starting point of two lines: one continued as RF3 (1991) in C, the other led to the commercial code FEFLOW [2]. The early RockFlow period is documented in textbooks on flow, mass and heat transport in fractured rock and in the monograph on computational methods in environmental fluid mechanics [4]. The step to GeoSys in 2001 introduced C++ and object orientation; OGS-5 (2008) became the first widely used open-source version. With OGS-6 (2015) the code was rewritten from scratch with a modern architecture and strict software-engineering practice, and it has followed the C++ standards since (C++14, C++17, C++20, C++23). The newest elements of the timeline are not simulator versions but tools around it: OpenWorkFlow and OGSTools for workflows, OGSGIScape for the integration of geographic data, and the current release series (OGS-6.9 in 2026, documentation for 6.5.9 linked in the lecture).

The list of features that accumulated along this path reads like a summary of the field: dynamic data structures, object orientation, high-performance computing, open source, software engineering, cross-platform builds, the DataExplorer, virtual reality, a Python framework, Jupyter integration, GIS-based workflows, automation and, most recently, AI-supported modelling. The images on the history slides (Mont Terri, EURAD, DECOVALEX, the GeomInt volume, THMC benchmark books) show that this development was driven by applications and international projects, not by software development for its own sake.

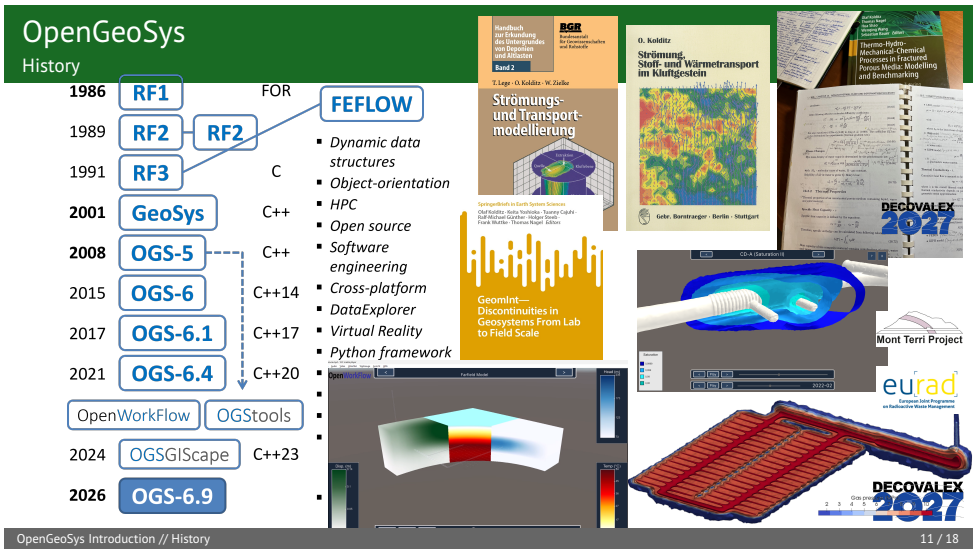


Figure 2: History of OpenGeoSys from RockFlow (1986) to OGS-6.9 (2026), with programming languages, key features and application projects (slide 11 of the lecture).

3.3 The Working Philosophy of OpenGeoSys

Figure 3 shows OGS as a layered core with interfaces around it.

Core and shell. At the centre is ogs^{THMC}, the C++ simulation kernel for coupled thermal, hydraulic, mechanical and chemical processes. Around it sits ogs^{Tools}, a Python layer that wraps

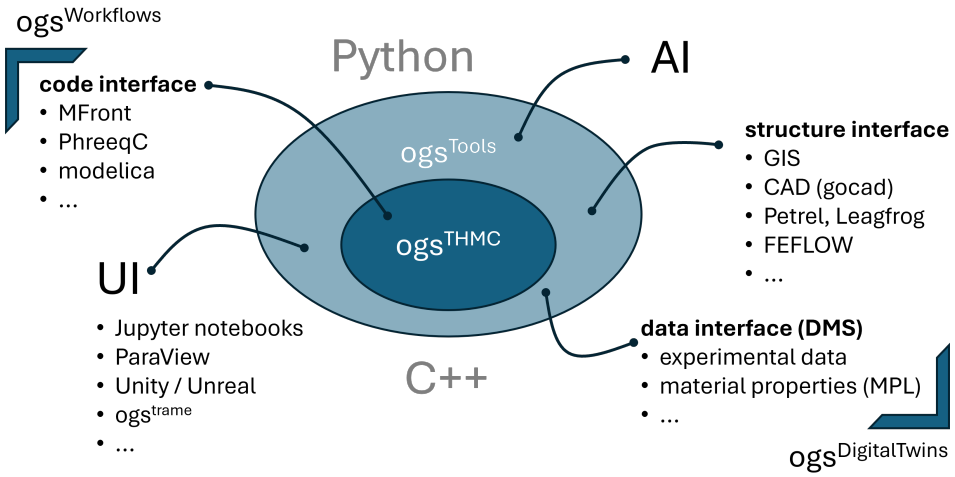


Figure 3: Working philosophy of OpenGeoSys: a high-performance C++ simulation core (ogs^{THMC}) embedded in a Python layer (ogs^{Tools}), connected to external codes, structural models, data, user interfaces and AI, and enabling ogs^{Workflows} and ogs^{DigitalTwins}.

the kernel and makes it accessible. The labels *Python* (top) and *C++* (bottom) show this split: high performance inside, flexible scripting outside.

Interfaces. Four groups of external connections link into the core or the Python layer:

- **Code interface:** couples external solvers and libraries directly to the kernel, e.g. MFront (constitutive models), PhreeqC (geochemistry) and Modelica (system models).
- **Structure interface:** brings in geometry and geological models from GIS, CAD (GOCAD), Petrel, Leapfrog and FEFLOW.
- **Data interface (DMS):** connects experimental data and material property definitions (MPL) for parameterisation, calibration and validation.
- **User interface (UI):** gives users ways to interact with and visualise models, including Jupyter notebooks, ParaView, Unity/Unreal and ogs^{trame} (web-based).

AI connects to the Python layer, so machine-learning methods such as surrogate models, optimisation or data-driven analysis can work alongside the simulations.

Overarching frames. The corner brackets mark the two application levels this setup supports. ogs^{Workflows} (top left) covers automated, reproducible modelling chains, mainly built on the code interfaces. ogs^{DigitalTwins} (bottom right) covers continuously updated, data-driven representations of real systems, mainly built on the data interfaces.

Philosophy. OGS combines a robust, efficient numerical core with an open, Python-based ecosystem. That makes it easy to connect to other tools, data and AI, so it was growing from a single simulator into a platform for workflows and digital twins.

4 The OGS ecosystem

The second half of the OGS introduction presents four components that a course participant will meet in the exercises.

Software engineering. OGS is developed with a continuous-integration (CI) system: every commit to the version control triggers builds, tests against benchmarks, code-coverage and profiling analyses and the generation of documentation and website [1]. Current developments include an OGS chatbot for writing project files (work in progress), Guix for reproducible software environments, AiiDA for provenance tracking, the integration of data from databases, and OGSTools for compact workflows.

OGSTools. OGSTools is a Python package that covers the full modelling chain around the simulator: pre-processing (meshing, project-file generation), simulation control and post-processing (plotting, comparison, convergence studies) [6]. For students it lowers the barrier considerably, because a complete OGS study can be written as one Python script or Jupyter notebook (`https://ogstools.opengeosys.org/stable/`).

Benchmark Gallery. The Benchmark Gallery (`https://www.opengeosys.org/6.5.9/docs/benchmarks/`) is organised by processes according to the THMC concept. Most entries are Jupyter notebooks that can be run on the ogs-binder server and contain the context and the complete workflow, typically based on OGSTools. The lecture recommends it as the starting point for one's own models: rather than writing a project file from scratch, one starts from a verified benchmark and modifies it step by step.

ModelHub. The ModelHub (`https://www.opengeosys.org/6.5.9/docs/hub/`) is a platform for joint benchmarking, initiated by the EURAD and DECOVALEX projects [5]. In contrast to the Benchmark Gallery it is organised by application scale, from laboratory to site, and is maintained as an interactive, git-based platform, so that different teams and codes can contribute and compare results.

5 Discussion

Read as a whole, the lecture makes three points that go beyond a software overview.

First, OpenGeoSys is presented as a *framework*, not as a single program. The relevant unit for the user is the workflow: data, mesh, project file, simulation, post-processing and documentation. The course organisation mirrors this, with Overleaf for notes and GitHub for exercises.

Second, the history shows that the code has evolved four decades because it was repeatedly rebuilt on the current standards of software engineering. The move from Fortran to C and C++, the rewrite of OGS-6, and the shift of attention from the numerical solver to workflows, provenance and reproducible environments are steps in the same direction.

Third, verification and benchmarking are the backbone of the framework. The CI system tests every change, the Benchmark Gallery provides verified starting points for users, and the ModelHub extends benchmarking to the international community. For the exercises this suggests a clear method: begin with a benchmark whose solution is known (Theis), then move to calibration against field data (STIMTEC), and only then to regional models with uncertain input (Selke catchment).

6 Conclusions | Take-home-messages

1. Lecture 0 gives the course a clear structure: three lecture blocks, exercises from installation to coupled field-scale problems, and the THMC tetrahedron as the common reference.
2. OpenGeoSys has developed over 40 years from the Fortran code RockFlow to a C++ framework with a Python ecosystem; its development has been driven by applications in geothermal energy, water resources, geotechnics and nuclear waste disposal.

3. The current focus is on the ecosystem around the simulator: CI-based software engineering, OGSTools, the Benchmark Gallery and the ModelHub.

4. For the course participants the practical message is to start from benchmarks, to work in scripted and versioned workflows, and to treat reproducibility and documentation as part of the modelling result.

5. AI provides great support for the entire workflow process, as OGS is a long-standing open-source project. OGS was one of the first numerical simulators for THMC processes, demonstrating the significant advantages of open-source software.

References

[1] L. Bilke et al. (2025). Cited in the lecture (slide 12) for the OGS software-engineering infrastructure.

[2] H.-J. G. Diersch (2014). *FEFLOW: Finite Element Modeling of Flow, Mass and Heat Transport in Porous and Fractured Media*. Springer, Berlin, Heidelberg. doi:10.1007/978-3-642-38739-5.

[3] V. A. Jiménez Martínez, *Hydraulic Changes Induced by Stimulation*, Doctoral thesis, Ruhr-Universität Bochum, Universitätsbibliothek, 2020. doi: 10.13154/294-7815.

[4] O. Kolditz (2002). *Computational Methods in Environmental Fluid Mechanics*. Springer, Berlin, Heidelberg. doi:10.1007/978-3-662-04761-3.

[5] O. Kolditz, D. Jacques, F. Claret, E. Holt, R. Szőke, D. Garcia, V. Montoya, S. V. Churakov, I. Szőke, A. Laikari, M. Chen, T. Zheng (2026). Digitalisation in geosciences for environmental protection. *Environmental Earth Sciences* 85(2), 51 (Editorial). doi:10.1007/s12665-025-12750-y.

[6] T. Meisel, F. Zill, J. Heinze, L. Bilke, M. Jäschke, F. Kiszkurno, N. Grunwald, T. Fischer, J. Buchwald (2026). OGSTools: A Python package for OpenGeoSys. *Journal of Open Source Software* 11(125), 10973. doi:10.21105/joss.10973.

2 Lecture 2: Continuum Mechanics

OpenGeoSys Training Korea/China 2026

Lecture 1: Continuum Mechanics
An Essay

Olaf Kolditz
Helmholtz Centre for Environmental Research – UFZ, Leipzig

7 October 2026

Abstract

The continuum-mechanics lecture of the OpenGeoSys (OGS) training course 2026 in Daejeon (Olaf Kolditz, Mehran Ghasabeh) lays the theoretical foundation for all following lectures and exercises. Starting from the THMC concept of OGS and the theory of porous media, it introduces the two ways of looking at a continuum, the Lagrangian (material) and the Eulerian (spatial) perspective, and connects them by the Reynolds transport theorem. With the Gauss–Ostrogradski theorem the integral conservation principle is turned into a general balance equation in integral and differential form. The flux of a conserved quantity is split into an advective part, carried by the mass-averaged velocity, and a diffusive part, described by Fick's law. The lecture closes with the three classical conservation quantities, mass, linear momentum and energy. This essay summarises the lecture and argues that its main message is economy of thought: one balance equation, specialised by choosing the conserved quantity, its flux and its source term, generates the governing equations of all THMC processes treated in the course.

1 Introduction

Numerical simulators such as OpenGeoSys solve partial differential equations, but the equations themselves are not invented for each application. They are derived from a small number of conservation principles of continuum mechanics, completed by constitutive laws for the material at hand. Lecture 1 of the OpenGeoSys training course, held in Daejeon on 7 October 2026, provides this common derivation. After a short repetition of the course overview from Lecture 0, it moves in a few steps from the physical picture of a porous medium to the general balance equation that underlies the hydromechanical and geomechanical lectures and exercises of the following days.

This essay follows the line of the lecture. Section 2 describes the porous medium as a multi-phase, multi-component continuum, Sect. 3 the Lagrangian and Eulerian perspectives, Sect. 4 the general balance equation, Sect. 5 the decomposition of fluxes and Sect. 6 the conservation quantities. Section 7 discusses what the lecture implies for the exercises.

2 The porous medium as a continuum

The lecture places continuum mechanics in the context of the THMC tetrahedron introduced in Lecture 0: thermal (T), hydraulic (H), mechanical (M) and chemical (C) processes correspond to applications in geothermal energy, geohydrology, geomechanics and reactive transport, and all four are described by balance equations of the same structure.

The object of these equations is a porous or fractured medium (Fig. 1). Its structure spans many scales, from boundary layers at the micrometre scale through single pores and fractures, local inhomogeneities and hydrogeological aquifer structures up to the regional aquifer system

at the kilometre scale. At the pore scale the medium consists of several *phases* γ (solid, liquid, gaseous), each of which may carry several *chemical components* k. A continuum description does not resolve this geometry. Instead, each phase and component is represented by averaged quantities defined at every point of the domain, and the total amount of a quantity is obtained by summation over components ($\sum_k$) and phases ($\sum_\gamma$). This is the point of view of the theory of porous media as developed by Bear [1, 2], Ehlers and Bluhm [5] and Coussy [3].

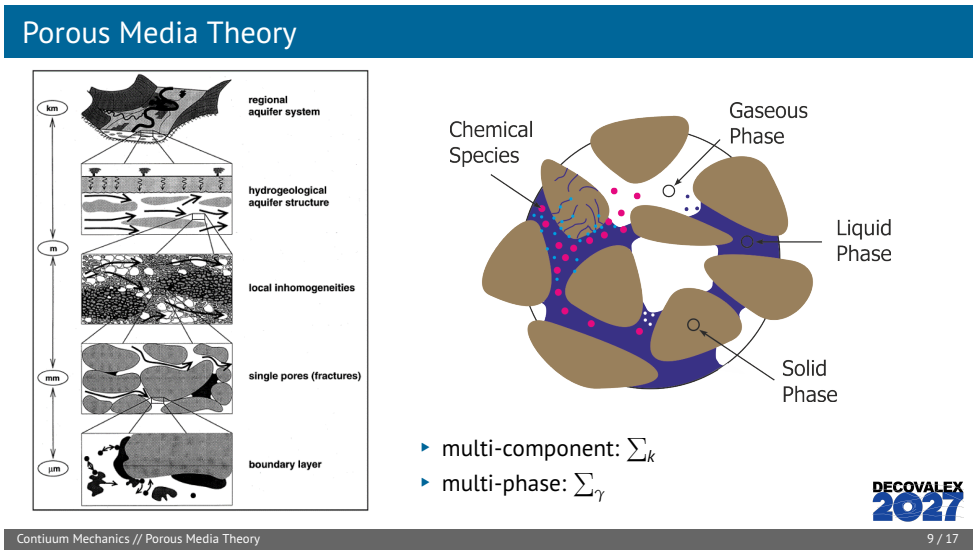


Figure 1: Porous media theory: scales of a porous or fractured aquifer (left) and the representation of a pore space with solid, liquid and gaseous phases and chemical species (right) (slide 9 of the lecture).

3 Lagrange and Euler perspectives

A continuum can be observed in two ways (Fig. 2). In the *Lagrangian* concept the observer follows a material volume $\Omega(t)$ along its pathline; the volume always contains the same particles, and its position and shape change with time. In the *Eulerian* concept the observer looks at a fixed control volume Ω_0 in space, through which the material flows. Solid mechanics usually prefers the Lagrangian view, fluid mechanics and transport the Eulerian view; porous media combine both, since the solid skeleton deforms while fluids flow through it.

Both views must yield the same conservation statement. The connection is the Reynolds transport theorem: the rate of change of a quantity ψ in a moving material volume equals its local rate of change in the fixed volume plus the net flux across the boundary,

$$\frac{\mathrm{d}}{\mathrm{d}t}\int_\Omega \psi \,\mathrm{d}\Omega = \int_{\Omega_0} \frac{\partial \psi}{\partial t}\,\mathrm{d}\Omega + \oint_{\partial\Omega_0} \psi\, \mathbf{v}\cdot \mathrm{d}\mathbf{S} = \int_\Omega q^\psi \,\mathrm{d}\Omega, \tag{1}$$

where q^ψ is the supply of ψ inside the volume. Equation (1) is the conservation principle in its most general form: what changes inside a volume is what flows in or out plus what is produced or consumed.

4 The general balance equation

The surface integral in Eq. (1) is converted into a volume integral by the Gauss–Ostrogradski (divergence) theorem,

$$\oint_{\partial\Omega_0} \mathbf{\Phi}^\psi \cdot \mathrm{d}\mathbf{S} = \int_{\Omega_0} \nabla\cdot\mathbf{\Phi}^\psi \,\mathrm{d}\Omega_0, \tag{2}$$

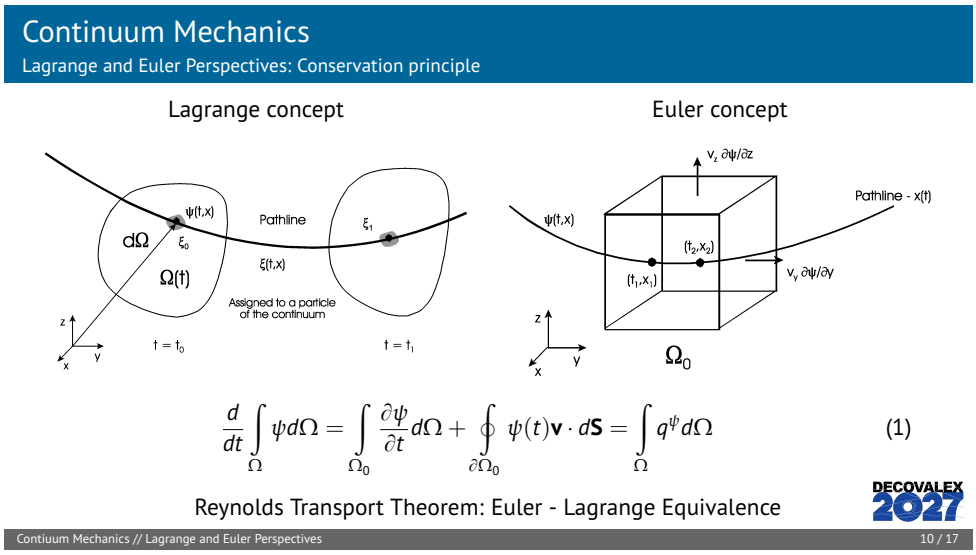


Figure 2: Lagrangian (material) and Eulerian (spatial) perspectives on a continuum, connected by the Reynolds transport theorem (slide 10 of the lecture).

where $\mathbf{\Phi}^\psi$ denotes the total flux of ψ. All terms are then integrals over the same domain, and because the balance must hold for every arbitrary volume, the integrands themselves must balance. This gives the general balance equation of continuum mechanics in differential form [6, 4],

$$\forall\, \Omega: \qquad \frac{\partial \psi}{\partial t} + \nabla \cdot \mathbf{\Phi}^\psi = q^\psi. \tag{3}$$

The step from the integral to the differential form requires that ψ and $\mathbf{\Phi}^\psi$ are sufficiently smooth. Where this is not the case, for instance across fractures or material interfaces, the integral form remains valid and leads to jump conditions. This is one reason why the finite-element method used in OpenGeoSys, which works with a weak (integral) form of the equations, is well suited for heterogeneous geological media.

5 Fluxes: from micro to macro

The lecture defines the total flux of ψ through the velocity $\mathbf{v}^E$ of the mixture, $\mathbf{\Phi}^\psi = \mathbf{v}^E \psi$. At the microscopic scale each constituent i moves with its own velocity $\mathbf{v}_i$. At the macroscopic scale only a mean velocity $\bar{\mathbf{v}}$ (the mass-averaged velocity) is resolved; the remaining part, $\hat{\mathbf{v}} = \mathbf{v}^E - \bar{\mathbf{v}}$, is the diffusion velocity of the constituent relative to the mean motion. The scatter plot on slide 12 illustrates this: individual particles move in different directions, whereas the averaged motion has a clear preferred direction.

Splitting the velocity splits the flux into two parts,

$$\mathbf{\Phi}^\psi = \mathbf{v}^E \psi = \underbrace{\bar{\mathbf{v}}\,\psi}_{\mathbf{\Phi}^\psi_A} + \underbrace{(\mathbf{v}^E - \bar{\mathbf{v}})\,\psi}_{\mathbf{\Phi}^\psi_D}, \tag{4}$$

an *advective* flux $\mathbf{\Phi}^\psi_A = \mathbf{v}\psi$, which carries ψ with the mean motion (with $\bar{\mathbf{v}} \equiv \mathbf{v}$), and a *diffusive* flux $\mathbf{\Phi}^\psi_D$. The diffusive flux cannot be computed from the macroscopic velocity and needs a constitutive law. The lecture uses Fick's law,

$$\mathbf{\Phi}^\psi_D = -\mathbf{D}^\psi \nabla \psi, \tag{5}$$

with a diffusion coefficient or tensor $\mathbf{D}^\psi$. Inserting both parts into Eq. (3) yields the advection–diffusion form of the general balance equation,

$$\frac{\partial \psi}{\partial t} + \nabla \cdot (\mathbf{v}\psi) - \nabla \cdot (\mathbf{D}^\psi \nabla \psi) = Q^\psi, \tag{6}$$

with a volumetric source term Q^ψ. Darcy's law for groundwater flow and Fourier's law for heat conduction have exactly the structure of Fick's law, with hydraulic conductivity and thermal conductivity in place of $\mathbf{D}^\psi$. Equation (6) is therefore the template for flow, heat and mass transport in the course.

6 Conservation quantities

The last step is to specify ψ. The amount of a quantity in a volume, $\Psi = \int_\Omega \psi \, \mathrm{d}\Omega$, is an *extensive* quantity, ψ the corresponding *intensive* (specific) quantity. Continuum mechanics knows three fundamental conservation quantities (Table 1).

Table 1: Extensive conservation quantities and their intensive counterparts.

Extensive quantity Ψ	Symbol	Intensive quantity ψ	Symbol
Mass	M	Mass density	ρ
Linear momentum	m	Linear momentum density	$\rho\mathbf{v}$
Energy	E	Energy density	$e = \rho i + \frac{1}{2}\rho v^2$

Each choice turns Eq. (3) into one of the governing equations: $\psi = \rho$ gives the continuity equation (mass balance), the basis of the hydraulic (H) process and of the Theis exercise; $\psi = \rho\mathbf{v}$ gives the momentum balance, which in the quasi-static limit becomes the stress equilibrium of the mechanical (M) process; and $\psi = e$, with internal energy i and kinetic energy $\frac{1}{2}v^2$, gives the energy balance of the thermal (T) process. Balances of component masses, written for each k, lead to the chemical (C) transport equations. For a porous medium, these balances are written for each phase γ and coupled through exchange terms, which is the origin of the coupled THMC problems that OpenGeoSys solves.

7 Discussion

Read together with the exercises, the lecture makes three points.

First, the general balance equation is a unifying principle. All processes of the THMC tetrahedron are instances of one equation that differ only in the conserved quantity, the flux law and the source term. This is reflected in the structure of OpenGeoSys itself, where processes share the same assembly and solution machinery and differ mainly in their constitutive models.

Second, the separation between balance laws and constitutive laws is fundamental. Equation (3) is exact; approximations enter through the flux laws (Fick, Darcy, Fourier) and through the averaging from pore to continuum scale. When a model fails to reproduce observations, as may happen in the STIMTEC exercise, it is usually the constitutive assumptions and their parameters, not the conservation principle, that need revision.

Third, the choice of perspective matters for coupled problems. Fluid flow is naturally described in Eulerian terms, the deformation of the solid skeleton in Lagrangian terms. Hydromechanics, the subject of Lecture 2, has to reconcile both, and the Reynolds transport theorem introduced here is the tool for doing so.

Some points of the slides could be clarified in a future revision: the differential form on slide 11 is a local balance written with a total time derivative on the left-hand side, the mixture velocity on slide 12 is written as a union rather than as a mass-weighted sum of constituent

velocities, and Fick's law on slide 13 is missing the argument of the gradient, $-\mathbf{D}^{\psi}\nabla\psi$. The section title "Contiuum" in the slide footer should read "Continuum".

8 Conclusions

1. Lecture 1 derives the general balance equation of continuum mechanics from the conservation principle, using the Reynolds transport theorem and the Gauss–Ostrogradski theorem.
2. A porous medium is treated as a multi-phase, multi-component continuum; balances are written for phases γ and components k and summed where needed.
3. The total flux is split into an advective part with the mass-averaged velocity and a diffusive part that requires a constitutive law such as Fick's law.
4. Choosing mass, momentum or energy as conservation quantity yields the governing equations of the H, M and T processes; component mass balances yield the C processes.
5. For the course participants the practical message is that every OGS process is a balance equation plus constitutive laws, and that model calibration addresses the constitutive part.

References

[1] J. Bear (1972). *Dynamics of Fluids in Porous Media*. Environmental Science Series. American Elsevier, New York.

[2] J. Bear (2018). *Modeling Phenomena of Flow and Transport in Porous Media*. Theory and Applications of Transport in Porous Media. Springer, Cham. doi:10.1007/978-3-319-72826-1.

[3] O. Coussy (2004). *Poromechanics*. 2nd ed. John Wiley & Sons, Chichester. doi:10.1002/0470092718.

[4] H.-J. G. Diersch (2014). *FEFLOW: Finite Element Modeling of Flow, Mass and Heat Transport in Porous and Fractured Media*. Springer, Berlin, Heidelberg. doi:10.1007/978-3-642-38739-5.

[5] W. Ehlers, J. Bluhm (eds.) (2002). *Porous Media: Theory, Experiments and Numerical Applications*. Springer, Berlin, Heidelberg. doi:10.1007/978-3-662-04999-0.

[6] O. Kolditz (2002). *Computational Methods in Environmental Fluid Mechanics*. Springer, Berlin, Heidelberg. doi:10.1007/978-3-662-04761-3.

3 Lecture 3: Hydromechanics and Groundwater Flow

OpenGeoSys Training Korea/China 2026

Lecture 2: Hydromechanics – Groundwater Flow
An Essay

Olaf Kolditz
Helmholtz Centre for Environmental Research – UFZ, Leipzig

7 October 2026

Abstract

The hydromechanics lecture of the OpenGeoSys (OGS) training course 2026 in Daejeon (Olaf Kolditz, Mehran Ghasabeh) applies the general balance equation of Lecture 1 to the hydraulic (H) process. With the density as conserved quantity the balance equation becomes the continuity equation. Its simplest special case, divergence-free flow of an incompressible, homogeneous fluid, is illustrated by an analytical source–sink solution and a short Python exercise. The lecture then writes the mass balance for the fluid in the pore space, closes it with Darcy's law and a storage relation, and obtains the groundwater flow equation in two equivalent forms: the hydraulic-head formulation of hydrogeology and the pressure formulation used in OpenGeoSys. This essay summarises the lecture and argues that its central message is the step from a conservation law to a solvable equation: continuity alone is not enough, and it is the constitutive laws for flux and storage that turn mass conservation into the diffusion-type equation solved in the Theis and STIMTEC exercises.

1 Introduction

Lecture 1 derived one general balance equation from which all THMC processes follow by choosing the conserved quantity, its flux and its source term. Lecture 2, held in Daejeon on 7 October 2026, makes the first such choice. It concentrates on the H corner of the THMC tetrahedron (Fig. 1), which covers water resources, water quality and geotechnical systems and is at the same time the hydraulic part of every coupled hydro-mechanical (HM) problem treated later in the course.

This essay follows the line of the lecture. Section 2 specialises the balance equation to mass conservation, Sect. 3 discusses divergence-free flow and the accompanying exercise, Sect. 4 derives the groundwater flow equation in the hydraulic-head formulation and Sect. 5 in the pressure formulation. Section 6 discusses what the lecture implies for the exercises.

2 Phase mass conservation

The starting point is the advection–diffusion form of the general balance equation from Lecture 1,

$$\frac{\partial \psi}{\partial t} + \nabla \cdot (\mathbf{v}\psi) - \nabla \cdot (\mathbf{D}^{\psi} \nabla \psi) = Q^{\psi}. \tag{1}$$

Choosing the density, $\psi = \rho$, gives the mass balance of a phase. A phase has no diffusive mass flux relative to its own mass-averaged velocity, because this velocity is defined such that it carries the total mass; hence $\mathbf{D}^{\rho} = 0$. Without sources the continuity equation in divergence form follows,

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) = 0. \tag{2}$$

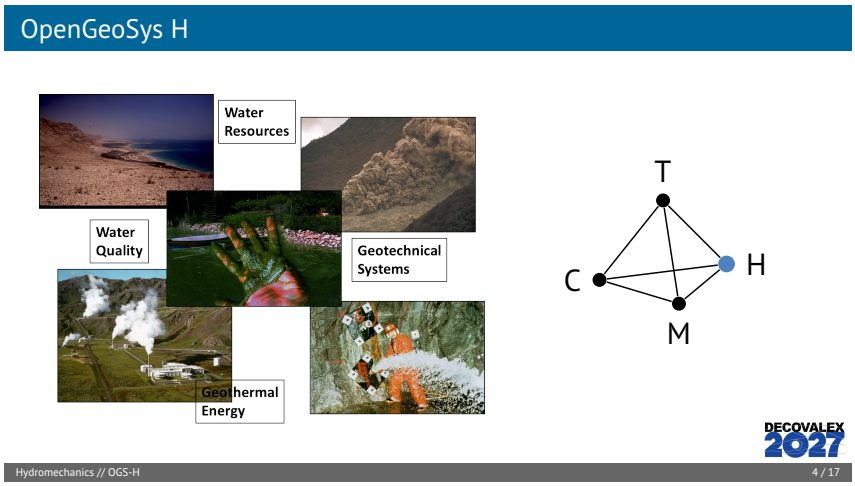


Figure 1: The hydraulic (H) process within the THMC concept of OpenGeoSys and its application fields (slide 4 of the lecture).

Expanding the divergence by the product rule and introducing the material derivative $\mathrm{d}/\mathrm{d}t = \partial/\partial t + \mathbf{v}\cdot\nabla$ gives the convective form

$$\frac{\partial \rho}{\partial t} + \mathbf{v}\cdot\nabla\rho + \rho\nabla\cdot\mathbf{v} = \frac{\mathrm{d}\rho}{\mathrm{d}t} + \rho\nabla\cdot\mathbf{v} = 0. \tag{3}$$

Both forms are mathematically equivalent for smooth fields. The divergence form, Eq. (2), is the one that is integrated in a finite-element or finite-volume scheme because it conserves mass exactly at the discrete level; the convective form, Eq. (3), is the one that reveals the physics: the density of a material particle changes only when the flow expands or contracts locally.

3 The simplest case: divergence-free flow

For an incompressible flow the density of each particle stays constant, $\mathrm{d}\rho/\mathrm{d}t = 0$, and Eq. (3) reduces to

$$\nabla\cdot\mathbf{v} = 0, \qquad \text{in 2D:} \qquad \frac{\partial v_x}{\partial x} + \frac{\partial v_y}{\partial y} = 0. \tag{4}$$

For a homogeneous fluid ($\rho = \text{const}$) this is the full continuity equation. A velocity field that satisfies Eq. (4) has no sources or sinks: whatever flows into a volume must flow out again.

The lecture gives an analytical example, the superposition of a source at $(L, 0)$ and a sink at $(-L, 0)$,

$$v_x = \frac{x-L}{(x-L)^2+y^2} - \frac{x+L}{(x+L)^2+y^2}, \qquad v_y = \frac{y}{(x-L)^2+y^2} - \frac{y}{(x+L)^2+y^2}. \tag{5}$$

Differentiation gives

$$\frac{\partial v_x}{\partial x} = \frac{y^2-(x-L)^2}{r_1^4} - \frac{y^2-(x+L)^2}{r_2^4}, \qquad \frac{\partial v_y}{\partial y} = \frac{(x-L)^2-y^2}{r_1^4} - \frac{(x+L)^2-y^2}{r_2^4}, \tag{6}$$

with $r_{1,2}^2 = (x \mp L)^2 + y^2$, so the two terms cancel and Eq. (4) holds. The cancellation holds everywhere *except* at the two singular points $r_1 = 0$ and $r_2 = 0$. There the divergence is a Dirac distribution of strength $\pm 2\pi$: the source and the sink are exactly the places where mass enters and leaves the domain. The example therefore shows two things at once: that divergence-free flow is a statement about the interior of a domain, and that wells (Theis exercise) enter the flow equation as point sources.

The exercise implements Eq. (5) in a dozen lines of Python with `numpy` and plots the field with `matplotlib.pyplot.quiver` (Fig. 2; script `EX22-divergenzfreie-stroemung.py` in the Hydroinformatik-2026 repository, after an example from the University of Auckland). The plot shows the typical dipole pattern, with streamlines leaving the source and converging on the sink.

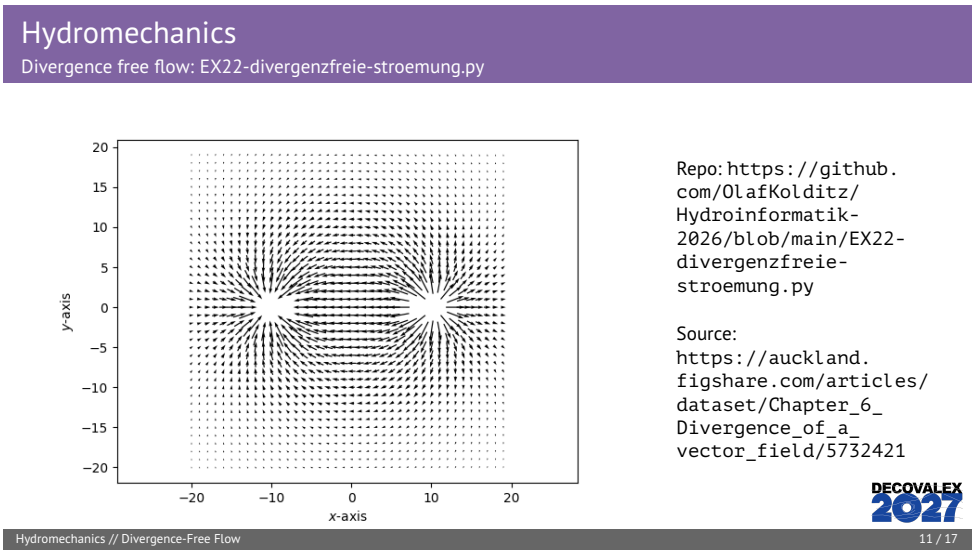


Figure 2: Divergence-free source–sink flow computed with the Python exercise (slide 11 of the lecture).

4 Groundwater flow: hydraulic-head formulation

In a porous medium the fluid occupies only the effective pore space n_e. The mass balance of the fluid phase, Eq. (1) with $\psi = n_e \rho$, reads

$$\frac{\partial (n_e \rho)}{\partial t} + \nabla \cdot (n_e \rho \mathbf{v}) = Q_\rho, \tag{7}$$

where $\mathbf{v}$ is now the pore (seepage) velocity. For an incompressible fluid, $\rho = \rho_w = \text{const}$, the density can be divided out, and the mass balance becomes a volume balance,

$$\frac{\partial n_e}{\partial t} + \nabla \cdot (n_e \mathbf{v}) = \frac{Q_\rho}{\rho_w} = Q. \tag{8}$$

This equation contains two unknowns, the porosity and the velocity, and it cannot be solved without two constitutive relations from groundwater hydraulics. The *storage relation* links the change of pore space to the change of hydraulic head h through the specific storage coefficient S_0, and *Darcy's law* links the specific discharge $\mathbf{q} = n_e \mathbf{v}$ to the head gradient through the hydraulic conductivity $\mathbf{K}$:

$$\frac{\partial n_e}{\partial t} = S_0 \frac{\partial h}{\partial t}, \qquad \mathbf{q} = n_e \mathbf{v} = -\mathbf{K} \nabla h. \tag{9}$$

Inserting both gives the groundwater flow equation in head formulation [1, 6, 4],

$$S_0 \frac{\partial h}{\partial t} - \nabla \cdot (\mathbf{K} \nabla h) = Q, \tag{10}$$

which, for a diagonal conductivity tensor, is written in Cartesian components as

$$S_0 \frac{\partial h}{\partial t} - \frac{\partial}{\partial x}\left(K_x \frac{\partial h}{\partial x}\right) - \frac{\partial}{\partial y}\left(K_y \frac{\partial h}{\partial y}\right) - \frac{\partial}{\partial z}\left(K_z \frac{\partial h}{\partial z}\right) = Q, \tag{11}$$

and for horizontal 2D flow the z-term is dropped. Equation (10) is a diffusion equation with hydraulic diffusivity K/S_0. It is the comparison of Eq. (10) with Eq. (1) that makes the point of the lecture: in groundwater flow the advective structure of the mass balance has become a diffusive one, because Darcy's law expresses the flux, like Fick's law, as a gradient of the state variable. In the steady, homogeneous case without storage, Eq. (10) reduces to the Laplace equation $\nabla^2 h = 0$, and the divergence-free flow of Sect. 3 reappears with $\mathbf{v} \propto -\nabla h$ as a potential flow.

5 Groundwater flow: pressure formulation

OpenGeoSys solves the same balance with the pore-water pressure p_w as primary variable. Expanding the time derivative in Eq. (7) with respect to pressure gives the mass balance formulation

$$\Big(\rho_w \frac{\partial n_e}{\partial p_w} + n_e \frac{\partial \rho_w}{\partial p_w}\Big)\frac{\partial p_w}{\partial t} - \nabla \cdot \Big(\rho_w \frac{\mathbf{K}_0}{\mu_w}(\nabla p_w - \rho_w \mathbf{g})\Big) = \rho_w W_0, \tag{12}$$

where the bracket collects the compressibility of the pore space and of the fluid, and Darcy's law is written with the intrinsic permeability $\mathbf{K}_0$ and the dynamic viscosity μ_w,

$$\mathbf{q} = -\frac{\mathbf{K}_0}{\mu_w}(\nabla p_w - \rho_w \mathbf{g}). \tag{13}$$

For constant fluid density this becomes the volume balance formulation with the OGS storage coefficient $\beta_S = \partial n_e / \partial p_w$,

$$\beta_S \frac{\partial p_w}{\partial t} - \nabla \cdot \Big(\frac{\mathbf{K}_0}{\mu_w}(\nabla p_w - \rho_w \mathbf{g})\Big) = W_0. \tag{14}$$

The two formulations are linked by $p_w = \rho_w g(h - z)$, which gives the conversions

$$\mathbf{K} = \frac{\rho_w g}{\mu_w}\mathbf{K}_0, \qquad S_0 = \rho_w g\, \beta_S. \tag{15}$$

The pressure formulation separates the properties of the rock ($\mathbf{K}_0$, β_S) from those of the fluid (ρ_w, μ_w). This is essential once density and viscosity vary with temperature or salinity (TH, HC) and once the pore pressure acts on the solid skeleton through effective stress (HM) [3, 5, 2]. The head formulation is the natural language of hydrogeology and of the Theis solution; the pressure formulation is the natural language of a coupled simulator. The main parameters are summarised in Table 1.

Table 1: Main parameters of the groundwater flow equation (after slide 16 of the lecture).

Symbol	SI unit	Definition	Formulation
h	m	hydraulic head	head
S_0	1/m	specific storage coefficient	head
$\mathbf{K}$	m/s	hydraulic conductivity (tensor)	head
p_w	Pa	pore-water pressure	pressure
$\beta_S = \partial n_e/\partial p_w$	1/Pa	storage coefficient of OGS	pressure
$\mathbf{K}_0$	m^2	intrinsic permeability (tensor)	pressure
μ_w	Pa s	dynamic fluid viscosity	pressure
ρ_w	kg/m^3	fluid density	both
n_e	–	effective porosity	both
Q, W_0	1/s	volumetric source/sink term	both

6 Discussion

Read together with the exercises, the lecture makes three points.

First, conservation alone does not determine the flow. The continuity equation constrains the velocity field but does not fix it; Eq. (4) has infinitely many solutions, of which the source–sink dipole is one. Only Darcy's law and the storage relation close the system and turn it into a well-posed problem for a single scalar, h or p_w. This continues the main message of Lecture 1: balance laws are exact, constitutive laws carry the physics of the material and its uncertainty.

Second, the type of the equation changes. The phase mass balance is a first-order transport equation; the groundwater flow equation is parabolic (transient) or elliptic (steady). This determines the numerical method, the boundary conditions required and the behaviour of the solution: a pressure perturbation spreads diffusively with diffusivity K/S_0, which is exactly what the Theis solution describes and what is observed in the STIMTEC pumping test in fractured rock.

Third, the choice of primary variable is a modelling decision. Head is convenient for homogeneous, isothermal freshwater aquifers; pressure is required as soon as the hydraulic process is coupled to heat, chemistry or mechanics. Course participants should therefore be able to convert parameters between both formulations, since field data are usually reported as K and S_0, while OpenGeoSys projects expect $\mathbf{K}_0$, μ_w and β_S.

Some points of the slides could be clarified in a future revision: the title slide says "Lecture 3" but the section slide and the course overview say "Lecture 2"; Eq. (8) on slide 7 should read $\partial v_x/\partial x + \partial v_y/\partial y = 0$; Eq. (4) on slide 6 should use the material derivative, $\mathrm{d}\rho/\mathrm{d}t = -\rho\nabla\cdot\mathbf{v}$, and Eq. (1) on slide 5 again writes the local balance with a total time derivative on the left-hand side; the source term on slide 13 switches from ρ_w to ρ_0; the parameter table on slide 16 lists n_e twice, once with the wrong unit 1/T; and with $\mathbf{g} = (0, 0, g)$ the term $\nabla p_w - \rho_w \mathbf{g}$ requires a downward-pointing z-axis, which should be stated. "Darcy' law" should read "Darcy's law". In the Python exercise the grid contains the points $(\pm L, 0)$, so the script divides by zero at the source and the sink; shifting the grid by half a cell avoids the warning and illustrates the singularity discussed above.

7 Conclusions

1. Lecture 2 specialises the general balance equation to mass conservation ($\psi = \rho$, $\mathbf{D}^\rho = 0$) and derives the continuity equation in divergence and convective form.
2. Divergence-free flow, $\nabla\cdot\mathbf{v} = 0$, is the continuity equation of an incompressible, homogeneous fluid; the source–sink example shows that sources and sinks appear as singularities of the field.
3. The fluid mass balance in the pore space, closed by Darcy's law and a storage relation, yields the groundwater flow equation $S_0\,\partial h/\partial t - \nabla\cdot(\mathbf{K}\nabla h) = Q$.
4. The equivalent pressure formulation used in OpenGeoSys separates rock and fluid properties and is the basis for coupled TH, HC and HM processes.
5. For the course participants the practical message is that the Theis and STIMTEC exercises solve a diffusion equation for head or pressure, whose behaviour is controlled by the ratio of conductivity to storage.

References

[1] J. Bear (1972). *Dynamics of Fluids in Porous Media*. Environmental Science Series. American Elsevier, New York.

[2] J. Bear (2018). *Modeling Phenomena of Flow and Transport in Porous Media*. Theory and Applications of Transport in Porous Media. Springer, Cham. doi:10.1007/978-3-319-72826-1.

[3] O. Coussy (2004). *Poromechanics*. 2nd ed. John Wiley & Sons, Chichester. doi:10.1002/0470092718.

[4] H.-J. G. Diersch (2014). *FEFLOW: Finite Element Modeling of Flow, Mass and Heat Transport in Porous and Fractured Media*. Springer, Berlin, Heidelberg. doi:10.1007/978-3-642-38739-5.

[5] W. Ehlers, J. Bluhm (eds.) (2002). *Porous Media: Theory, Experiments and Numerical Applications*. Springer, Berlin, Heidelberg. doi:10.1007/978-3-662-04999-0.

[6] O. Kolditz (2002). *Computational Methods in Environmental Fluid Mechanics*. Springer, Berlin, Heidelberg. doi:10.1007/978-3-662-04761-3.

4 Exercise 1: Theis: Wellbore Hydraulics

The Theis Problem Revisited: From the Analytical Well Function to Leaky 2D Simulations with OpenGeoSys

Olaf Kolditz Mehran Ghasabeh
Helmholtz Centre for Environmental Research – UFZ, Leipzig

OpenGeoSys Training – Korea/China, Daejeon, 8 October 2026

Abstract

The Theis solution for a pumping well in a confined aquifer is the classical entry point into transient groundwater hydraulics and a standard verification case for numerical simulators. This short essay summarises Exercise 1 of the OpenGeoSys (OGS) training course: the analytical Theis solution, its numerical reproduction in an axially symmetric and a two-dimensional circular model, and an extension towards matrix leakage with constant and time-dependent sink terms. The leakage is finally re-implemented through the OGS Python interface, which reproduces the XML-based results to machine precision and opens the way to head-dependent (Hantush-type) leakage. The exercise illustrates how a simple benchmark becomes a vehicle for learning model set-up, mesh handling, boundary conditions, scripting and the interpretation of deviations from idealised theory.

1 Why Theis?

In 1935 C. V. Theis transferred the heat-conduction analogy to groundwater flow and derived the transient drawdown around a well pumping at a constant rate from a confined aquifer. Ninety years later the solution is still the backbone of pumping-test analysis [2] and the first benchmark any groundwater code must pass [1]. Its value for teaching lies in its idealisation: the aquifer is homogeneous, isotropic, confined, of uniform thickness and infinite radial extent; the well fully penetrates the aquifer, discharges at a constant rate and has negligible storage; no other wells or regional trends interfere. Each numerical model in the exercise relaxes, or is forced to relax, one of these assumptions – and the results show exactly what that costs.

2 Analytical solution

The drawdown s at distance r and time t is

$$s(r,t) = h_0 - h(r,t) = \frac{Q}{4\pi T}\,W(u), \qquad u = \frac{r^2 S}{4Tt}, \tag{1}$$

with the well function (exponential integral)

$$W(u) = -\gamma - \ln u + \sum_{k=1}^{\infty} \frac{(-1)^{k+1} u^k}{k\,k!} \;\approx\; -\gamma - \ln u \quad (u \ll 1), \tag{2}$$

where Q is the pumping rate, T the transmissivity, S the storage coefficient and $\gamma \approx 0.5772$ the Euler–Mascheroni constant. The truncated form is the Cooper–Jacob approximation: for late times the drawdown grows linearly in $\ln t$, which appears as straight lines in the semi-logarithmic time plots below.

3 Numerical models

3.1 Axially symmetric model

The first model exploits the radial symmetry of the problem. A one-dimensional radial mesh is flagged as `axially_symmetric` in the OGS project file, and the `LiquidFlow` process solves the transient pressure (head) equation. The comparison with Eq. (1) reproduces the benchmark of the OGS gallery and confirms that the radial discretisation and the time stepping are adequate.

3.2 Two-dimensional circular model

The second model resolves the aquifer as a full disk of radius $R = 304.8\,\mathrm{m}$ (1000 ft) triangulated with `gmsh` (3055 nodes, 6012 triangles). Two practical steps are part of the exercise: vertex (zero-dimensional) elements produced by `gmsh` must be removed, since OGS cannot handle them, and the circular boundary must be extracted as a separate line mesh so that a boundary condition can be assigned to it. The project therefore loads two meshes – domain and boundary – and places the well as a nodal source term at the origin via a GML point.

Table 1: Parameters of the 2D model (`ogs-basic.prj`). With unit density and viscosity and no gravity, pressure is identical to hydraulic head and the permeability acts as transmissivity of an aquifer of unit thickness.

Quantity	Value	Remark
Transmissivity T	$9.2903 \times 10^{-4}\,\mathrm{m^2/s}$	$= 0.01\,\mathrm{ft^2/s}$
Storage coefficient S	1×10^{-3}	
Well rate Q	$1.6 \times 10^{-2}\,\mathrm{m^3/s}$	nodal source at $r = 0$
Outer boundary	$h = 0$ at $R = 304.8\,\mathrm{m}$	Dirichlet
Simulated time	$8.64 \times 10^5\,\mathrm{s}$ (10 d)	Backward Euler, 66 steps

Note that the well is implemented as an *injection* (positive source). Because the problem is linear, the computed head rise is the mirror image of the drawdown of a pumping well, and it is plotted as such.

The 2D results agree with Theis at early times but deviate systematically later: whereas the analytical drawdown keeps growing with $\ln t$, the numerical curves flatten and reach a steady state. This is not a numerical error but the consequence of the finite domain with a fixed-head boundary, which violates the “infinite aquifer” assumption. The late-time limit is the Thiem solution

$$s_\infty(r) = \frac{Q}{2\pi T} \ln \frac{R}{r}, \tag{3}$$

which for the observation point at the well radius $r = 0.3048\,\mathrm{m}$ gives $s_\infty \approx 2.74\,\mathrm{m} \times \ln 1000 \approx 18.9\,\mathrm{m}$ – precisely the plateau seen in the simulations. The time at which the curves leave the Theis line marks the arrival of the cone of depression at the boundary.

3.3 Matrix leakage

The third model adds a distributed sink representing leakage from the aquifer into the surrounding matrix (or an aquitard). A third mesh, `mesh_leakage.vtu`, covers the domain and carries references to the bulk mesh (`bulk_node_ids`, generated with `identifySubdomains`). The leakage is a volumetric source term:

```
<source_term>
    <mesh>mesh_leakage</mesh>
    <type>Volumetric</type>
    <parameter>leakage_term</parameter>
```

```
</source_term>
...
<parameter>
    <name>leakage_term</name>
    <type>Constant</type>
    <value>-1e-7</value>
</parameter>
```

A uniform sink q_L in a disk with fixed boundary head superposes a parabolic depression $q_L(R^2 - r^2)/(4T)$ on the Thiem cone. For $q_L = -10^{-7}\,\mathrm{s}^{-1}$ this lowers the steady head by about 2.5 m at the centre; for $-2 \times 10^{-7}\,\mathrm{s}^{-1}$ by about 5 m. Integrated over the disk area ($\approx 2.9 \times 10^5\,\mathrm{m}^2$), the leakage of the first case ($\approx 0.029\,\mathrm{m}^3/\mathrm{s}$) already exceeds the well rate, so the boundary supplies the difference. In the second case the heads near the boundary become negative: the matrix withdraws more water than the well delivers. Leakage thus reduces the late-time drawdown and shortens the time needed to reach steady state – the same qualitative behaviour that the Hantush–Jacob leaky-aquifer solution predicts.

3.4 Time-dependent leakage

Next, the constant sink is replaced by a function parameter,

$$q_L(t) = -\frac{10^{-5}}{\sqrt{t}}, \tag{4}$$

mimicking diffusive exchange with a matrix whose near-fracture zone gradually equilibrates (`ogs-time-dependent-source-term.prj`). Comparing with the constant case reveals a crossover: the two rates are equal at $\sqrt{t} = 100$, i.e. $t = 10^4$ s. Before that the time-dependent leakage is stronger and the heads are lower; afterwards it decays below $10^{-7}\,\mathrm{s}^{-1}$ and the solution moves back towards the leakage-free case (Fig. 1).

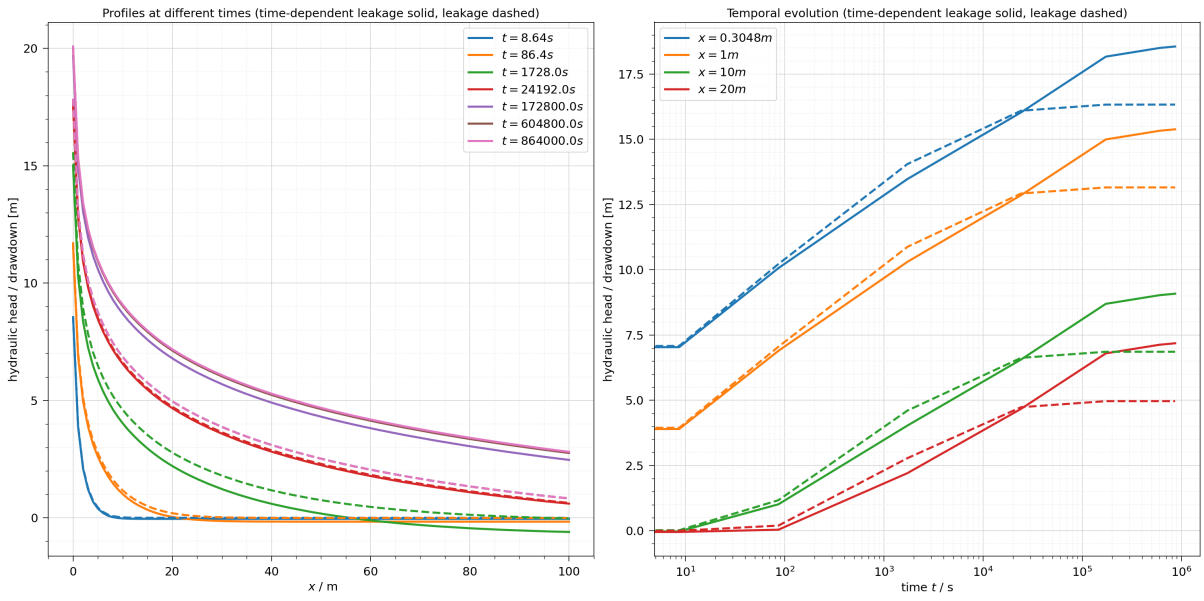


Figure 1: Constant ($q_L = -10^{-7}\,\mathrm{s}^{-1}$, dashed) versus time-dependent ($q_L = -10^{-5}/\sqrt{t}$, solid) leakage. Left: head profiles along the x-axis at selected times; right: head evolution at $x = 0.3048, 1, 10, 20$ m.

3.5 Leakage via the OGS Python interface

XML parameters are convenient for constants and closed-form expressions, but they cannot depend on the solution itself. OGS therefore offers a Python interface for boundary conditions and source terms: the project file names a Python script, and a source term of type `Python` refers to an object defined in that script. In `ogs-leakage-python.prj` the volumetric source term and the `leakage_term` parameter are replaced by

```
<python_script>leakage_source_term.py</python_script>
...
<source_term>
    <!-- leakage defined in leakage_source_term.py -->
    <mesh>mesh_leakage</mesh>
    <type>Python</type>
    <source_term_object>leakage</source_term_object>
</source_term>
```

The `<python_script>` entry is placed at the top level of the project file (after `<geometry>`); its path is resolved relative to the project file. The script defines a class derived from `SourceTerm` with a single method `getFlux(t, coords, primary_vars)`, which OGS calls at every integration point of `mesh_leakage`. It returns the source value and its derivatives with respect to the primary variables. OGS integrates $\int_\Omega N^\mathsf{T} q_L \, \mathrm{d}\Omega$ into the right-hand side, so the Python source term acts exactly like the former volumetric one. The core of `leakage_source_term.py` reads:

```
import math
try: # OGS >= 6.5 (wheel / embedded interpreter)
    import ogs.callbacks as OpenGeoSys
except ModuleNotFoundError: # older OGS versions
    import OpenGeoSys

MODE = "constant" # "constant" | "time" | "hantush"
Q_CONST = -1.0e-7 # constant leakage rate [1/s]
A_TIME = -1.0e-5 # amplitude of 1/sqrt(t) leakage
LEAKANCE = 1.0e-7 # aquitard leakance K'/b'
P_REF = 0.0 # head above the aquitard

class LeakageSourceTerm(OpenGeoSys.SourceTerm):
    def getFlux(self, t, coords, primary_vars):
        p = primary_vars[0]
        dq_dp = 0.0
        if MODE == "constant":
            q = Q_CONST
        elif MODE == "time":
            q = A_TIME / math.sqrt(t) if t > 0.0 else 0.0
        elif MODE == "hantush":
            q = -LEAKANCE * (p - P_REF)
            dq_dp = -LEAKANCE
        return (q, [dq_dp]) # value, d(value)/d(primary vars)

leakage = LeakageSourceTerm() # referenced by <source_term_object>
```

Recent OGS releases (6.5 and later) provide the module as `ogs.callbacks`; the import tries this name first and falls back to `OpenGeoSys` for older versions. The three modes cover the leakage models of this exercise:

- `constant`: $q_L = -10^{-7}\,\mathrm{s}^{-1}$, identical to `ogs-leakage.prj`;
- `time`: $q_L = -10^{-5}/\sqrt{t}$, identical to `ogs-time-dependent-source-term.prj`;
- `hantush`: $q_L = -L\,(p - p_\mathrm{ref})$, a head-dependent leakage through an aquitard of leakance $L = K'/b'$ as in the Hantush–Jacob leaky-aquifer model.

Verification. With OGS 6.5.9 the `constant` and `time` modes reproduce the XML-based simulations to machine precision: the largest nodal pressure difference over all output times is about 10^{-13} (Table 2).

Table 2: Comparison of the Python source term with the XML-based leakage models: maximum head and maximum nodal difference at selected output times.

Case	t [s]	$\max \lvert p \rvert$ [m]	$\max \lvert \Delta p \rvert$ [m]
`constant` vs. `ogs-leakage.prj`	1.728×10^5	17.855	6.9×10^{-14}
	8.64×10^5	17.856	2.8×10^{-14}
`time` vs. `ogs-time-dependent-source-term.prj`	1.728×10^5	19.693	7.1×10^{-14}
	8.64×10^5	20.082	3.2×10^{-14}

Head-dependent leakage needs Newton. The `hantush` mode is the real gain of the Python interface, but it changes the numerics. With the Picard solver of the original project the leakage is evaluated with the pressure of the previous iteration; the derivative $\partial q_L / \partial p$ is not used. The resulting fixed-point iteration converges only if $L\,\Delta t / S$ is sufficiently small. In this case ($L = 10^{-7}$, $S = 10^{-3}$) it already fails at $\Delta t = 7776\,\mathrm{s}$ ($L\,\Delta t/S \approx 0.8$), and the run aborts at $t \approx 1.6 \times 10^4\,\mathrm{s}$. The companion project `ogs-leakage-python-newton.prj` therefore switches to a Newton solver with a numerical (central-differences) Jacobian for the bulk process; the Python source term contributes the analytical derivative `dq_dp`:

```
<process>
   ...
   <jacobian_assembler>
      <type>CentralDifferences</type>
      <epsilons>1e-6</epsilons>
   </jacobian_assembler>
</process>
...
<nonlinear_solver>
   <name>basic_newton</name>
   <type>Newton</type>
   <max_iter>10</max_iter>
   <linear_solver>general_linear_solver</linear_solver>
</nonlinear_solver>
```

Because the Jacobian is no longer guaranteed to be symmetric, the Eigen solver is changed from CG to BiCGSTAB with ILUT preconditioning. Newton converges in about two iterations per time step. In the `constant` mode it gives the same result as Picard ($\max \lvert \Delta p \rvert \approx 3 \times 10^{-14}$). With $L = 10^{-7}$ the Hantush-type leakage changes the heads by up to about 0.5 m compared with the constant sink. This value of L is illustrative; for a quantitative comparison with the Hantush solution it has to be derived from the aquitard properties.

3.6 Running the Python version

The Python source term requires an OGS build with Python support. The simplest way is the OGS Python wheel, which contains the `ogs` executable with an embedded interpreter and the `ogs.callbacks` module:

```
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install ogs # tested with ogs 6.5.9
```

The working directory contains the project file, the Python script and the meshes and geometry it references:

```
leakage/
  ogs-leakage-python.prj # Picard: modes "constant", "time"
  ogs-leakage-python-newton.prj # Newton: all modes, required for "hantush"
  leakage_source_term.py # getFlux(), select MODE here
  mesh_domain.vtu mesh_boundary.vtu mesh_leakage.vtu
  well_injection.gml
```

The leakage model is selected by editing `MODE` (and, if required, the parameters) at the top of `leakage_source_term.py`. OGS does not create missing output directories, so the folder named in `<prefix>` has to exist before the run:

```
cd leakage
mkdir ../results_leakage_python # Windows: mkdir ..\results_leakage_python
ogs ogs-leakage-python.prj > ogs.log

# head-dependent leakage (MODE = "hantush")
mkdir ../results_leakage_python_newton
ogs ogs-leakage-python-newton.prj > ogs_newton.log
```

The simulation can also be run from a Python session or a Jupyter notebook, for instance as part of a parameter study on the leakance:

```
from ogs import OGSSimulator

sim = OGSSimulator.OGSSimulation(["", "ogs-leakage-python.prj"])
sim.execute_simulation()
sim.close()
```

The results (`theis2Dcircle_ts_*.vtu` and a `.pvd` collection) are post-processed in the same way as the XML-based runs, e.g. with ParaView or `ogstools`. Typical problems are visible directly in the log file:

- `No module named 'OpenGeoSys'`: the script uses the old module name with a new OGS version; use the `try/except` import above.
- `could not open file .../theis2Dcircle.pvd`: the output directory does not exist.
- `The nonlinear solver failed`, followed by `cannot reduce the time step`: a head-dependent source term run with Picard; use the Newton project.

4 Lessons learned

The Theis exercise is small, but it touches most of the workflow of a real groundwater study:

- *Verification needs matching assumptions.* Agreement with Theis holds only while the cone of depression has not reached the model boundary; afterwards Thiem is the correct reference.
- *Meshes are more than geometry.* Removing vertex elements, extracting boundaries and linking submeshes to the bulk mesh are recurring pre-processing steps in OGS.
- *Source terms are flexible.* Nodal, volumetric and function-based parameters allow wells and leakage to be added with a few lines of XML, and simple superposition arguments help to check the results quantitatively.
- *Python source terms add physics and responsibility.* A Python function can depend on time, space and the solution itself. Once the source depends on the primary variable, its derivative and the choice of nonlinear solver matter: Picard may diverge where Newton converges in two iterations.

A natural next step is to connect the Python leakage term to physically based matrix-diffusion models, and to verify the `hantush` mode against the Hantush–Jacob solution, as used in workflows for layer reduction and efficient geo-hydraulic modelling [3].

Remark on the course material. The slide listing for the time-dependent leakage shows `-1e-7/sqrt(t)`, whereas the project file and the bullet list use `-1e-5/sqrt(t)`; the essay follows the project file. In `plot_compare_numerical_solutions.py` the variable `ms_basic` actually loads the time-dependent leakage results.

References

[1] O. Kolditz. *Computational Methods in Environmental Fluid Mechanics*. Springer, Berlin Heidelberg, 2002. doi:10.1007/978-3-662-04761-3.

[2] G. P. Kruseman and N. A. de Ridder. *Analysis and Evaluation of Pumping Test Data*, 2nd ed. ILRI Publication 47, Wageningen, 1991.

[3] E. Nixdorf, M. Sips and P. Morstein. Development of a digital workflow for layer reduction strategies and visual-analytical outcome analysis to enhance geo-hydraulic modelling efficiency. *Digital Water*, 3(1):1–24, 2025. doi:10.1080/28375807.2025.2460825.

5 Exercise 2: STIMTEC: Hydraulic Injection Tests - Single Fracture Model

Evaluation of the STIMTEC Hydraulic Injection Tests BH10_20180716_51.6 and BH10_20180718_40.6

Single-Fracture Model: Calibration of Permeability and Storage

Olaf Kolditz*
Helmholtz Centre for Environmental Research – UFZ, Leipzig

3 October 2026

Abstract

Two stepwise hydraulic injection tests in borehole BH10 of the STIMTEC experiment, 51.6 (13 main rate steps up to 10 l/min, maximum pressure build-up 6.7 MPa) and 40.6 (six rate steps up to 8 l/min, 3.8 MPa), were evaluated with a single-fracture model: a planar $100\,\mathrm{m} \times 100\,\mathrm{m}$ fracture with a point injection, solved with OpenGeoSys 6.5.9 (`LIQUID_FLOW`). Permeability k and specific storage S were calibrated against the digitised pressure records in five model variants. With constant k and S (V1, V2) the measurements cannot be reproduced (RMSE 1.69 MPa for 51.6, 1.16 MPa for 40.6), because the measured pressure increases much less than proportionally with the injection rate ($\Delta p \propto q^{0.47}$ and $q^{0.31}$). If permeability and storage increase with the injection rate (V3, V4) the fit improves by a factor of 5 (51.6, RMSE 0.32 MPa) and 12 (40.6, RMSE 0.098 MPa). The calibrated permeability rises from $3.4 \cdot 10^{-15}$ to $2.4 \cdot 10^{-14}\,\mathrm{m}^2$ (factor 7) in test 51.6 and from $6.2 \cdot 10^{-16}$ to $1.7 \cdot 10^{-14}\,\mathrm{m}^2$ (factor 28) in test 40.6, which indicates hydraulic opening of the fracture. A pressure-dependent formulation via the OpenGeoSys Python interface (V5b) reproduces the trend but not the fit quality, because of the explicit coupling. The two tests differ strongly in storage (40.6: 15–50 times larger) and hence in hydraulic diffusivity ($\approx$ 2–140 m^2/s vs. $\approx 0.02\,\mathrm{m}^2/\mathrm{s}$), which controls whether the model boundary influences the result.

*This report, which is based on a systematic OGS study of the STIMTEC experiment, was supported by AI (Claude-Opus 5.5)

Contents

1 Introduction **3**

2 Field tests and data **3**
2.1 Digitisation . . . 3
2.2 Injection steps . . . 3
2.3 Pressure–rate relation . . . 3

3 Single-fracture model **5**
3.1 Geometry and mesh . . . 5
3.2 Governing equation and parameters . . . 5
3.3 Initial and boundary conditions . . . 6
3.4 Discretisation and solver . . . 6

4 Calibration method **6**
4.1 Objective function . . . 6
4.2 Model variants . . . 7
4.3 Optimisation . . . 7

5 Results: test 51.6 **7**

6 Results: test 40.6 **9**

7 Comparison and discussion **10**
7.1 Calibrated parameters . . . 10
7.2 Verification . . . 12

8 Conclusions and recommendations **12**

A Files and workflow **13**

1 Introduction

The STIMTEC experiment investigates hydraulic stimulation of crystalline rock at the decametre scale. In borehole BH10, stepwise injection tests were carried out in packed intervals; their pressure and flow-rate records allow the hydraulic properties of the stimulated fracture(s) to be derived. This report documents the evaluation of two of these tests, BH10_20180716_51.6 and BH10_20180718_40.6 (in the following "51.6" and "40.6"), with the simplest structural model, a single planar fracture that contains the injection point.

The objectives were to find permeability and storage values for which the computed fluid pressure at the injection point best matches the measured pressure build-up, and to identify which model assumptions are needed for an adequate fit. Starting from constant properties, the model was extended step by step: a pressure boundary condition, permeability and storage coupled to the injection rate, a point-like boundary, and a pressure-dependent formulation through the OpenGeoSys Python interface. Both tests were evaluated with the same workflow; test 51.6 was analysed first (26 September 2026), test 40.6 followed (30 September 2026). An extension to a discrete fracture network (DFN) model is reported separately.

2 Field tests and data

2.1 Digitisation

Only graphical records of the tests were available, so pressure and rate had to be digitised.

- **51.6:** source is a raster image of an Excel chart (`BH10_20180716_51.6.png`) with pressure [MPa] and injection rate [l/min] on a common axis. The curves were extracted pixel-wise by colour (`digitize_bh10.py`, `digitize_bh10_51_rate.py`; rate: median of the coloured pixels per column, moving median over 15 s). On 2 October 2026 the vertical calibration was corrected: the zero line of the chart is the darker axis line and had not been detected as a grid line, which shifted the raw values by −2 MPa. The pressure build-up Δp changes by at most 3 kPa; all calibrations remain valid (verified, Sect. 7.2).
- **40.6:** source is a vector PDF (`BH10_20180718_40.6.pdf`, Matplotlib). The curves were read directly from the PDF paths (`digitize_bh10_40.py`, pdfplumber) with axes calibrated via the tick marks, so no pixel digitisation was needed.

The baseline pressure is the median before the start of injection: 0.150 MPa ($t < 2650$ s, 51.6) and 0.087 MPa ($t < 1780$ s, 40.6). All comparisons refer to the pressure build-up $\Delta p = p - p_{\text{base}}$ (Fig. 1).

2.2 Injection steps

The injection is represented in the models by piecewise constant rates (Tables 1 and 2). For 40.6 the steps were derived from the digitised rate: step boundaries from the jumps of the smoothed rate, step values as the mean of the raw rate (`injektion_stufen_40.py`). For 51.6 the step function of the original project file was used; the rate digitised later (`injektion_stufen_51.py`) confirms it for the main steps but shows the boundaries 4–6 s earlier and differences in three windows (pre-injection rate, 2706 s to 2800 s and 2942 s to 3081 s).

2.3 Pressure–rate relation

Both tests share a characteristic feature that governs the whole evaluation: after each rate step the pressure rises quickly and then flattens, and the late-step pressure grows much less than proportionally with the rate (Fig. 2). A power-law fit gives $\Delta p \propto q^{0.47}$ for 51.6 and $\Delta p \propto q^{0.31}$ for 40.6. Over the test, the rate increases by a factor of ≈ 16 (51.6) and 14.6 (40.6), the pressure

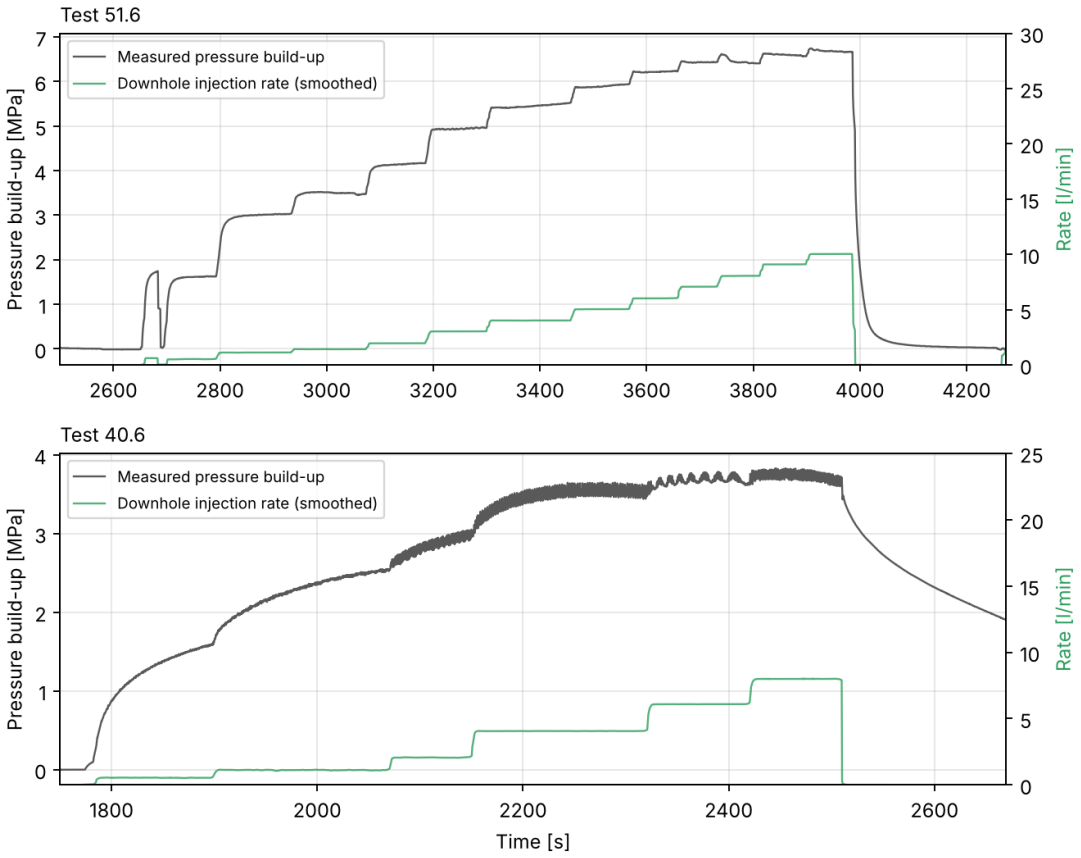


Figure 1: Digitised pressure build-up and downhole injection rate of tests 51.6 (top) and 40.6 (bottom).

Table 1: Injection steps of test 51.6: model (project file) and digitised rate.

	Model (`stimtec*.prj`)		Digitised	
Step	until t [s]	q [l/min]	until t [s]	q [l/min]
0	2663.9	0.062	2659.0	0.001
1	2689.3	0.252	2683.0	0.627
–	2706.5	0.063	2700.8	0.019
2	2800.1	0.163	2795.5	0.529
3	2942.2	1.322	2936.2	1.128
4	3081.1	1.643	3077.0	1.430
5	3195.5	2.066	3189.2	1.960
6	3308.3	3.070	3303.2	3.039
7	3466.0	4.168	3461.5	4.028
8	3575.3	5.227	3571.0	5.041
9	3665.6	6.000	3661.2	6.028
10	3740.8	6.716	3736.2	7.082
11	3819.2	7.873	3815.5	8.058
12	3908.8	8.813	3902.0	9.099
13	3991.4	9.694	3988.2	10.039
shut-in	4150.0	0.059	4259.6	0.005

Table 2: Injection steps of test 40.6 (model = digitised, `injektion_stufen_40.csv`).

Step	until t [s]	q [l/min]	q [m^3/s]
0 (pre-injection)	1785.0	0.003	$4.55 \cdot 10^{-8}$
1	1900.5	0.548	$9.13 \cdot 10^{-6}$
2	2071.5	1.118	$1.86 \cdot 10^{-5}$
3	2151.8	2.066	$3.44 \cdot 10^{-5}$
4	2322.8	4.061	$6.77 \cdot 10^{-5}$
5	2422.3	6.086	$1.01 \cdot 10^{-4}$
6	2510.3	7.994	$1.33 \cdot 10^{-4}$
shut-in	2669.8	0.011	$1.86 \cdot 10^{-7}$

only by ≈ 2.2 and 2.3. For Darcy flow with constant properties the relation would be linear. The data therefore point to a transmissivity that increases with the injection rate or pressure.

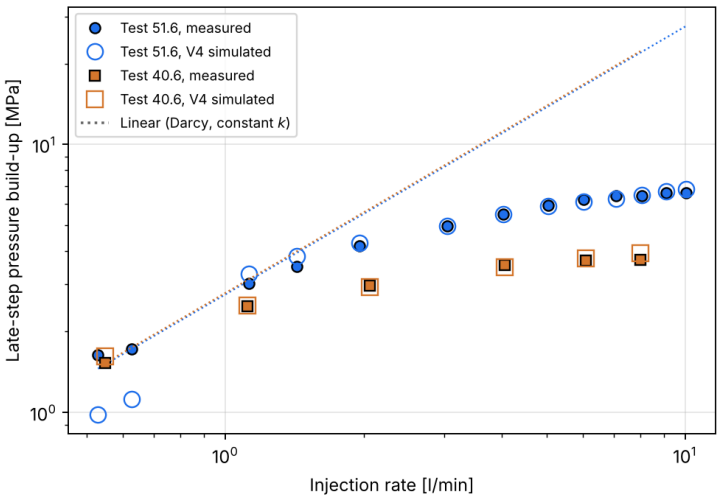


Figure 2: Mean measured pressure build-up in the last third of each injection step versus rate (filled symbols), with the calibrated model V4 (open symbols) and the linear relation expected for constant k (dotted). For 51.6 the two lowest steps of the model use the rates of the project file (Table 1).

3 Single-fracture model

3.1 Geometry and mesh

The fracture is represented as a planar square of 100 m × 100 m (`stimtec-square.vtu`, geometry `stimtec-square.gml`) with the injection point "well" at its centre (Fig. 3). The mesh has 939 nodes, 1796 linear triangles and 80 boundary line elements; the mean edge length is 3.5 m, refined to 1.5 m within 5 m of the well (minimum 0.86 m). The fracture is treated as a 2D continuum of unit thickness, so k and S are equivalent values for a 1 m thick layer; the injection volume is introduced at a single node.

3.2 Governing equation and parameters

The OpenGeoSys `LIQUID_FLOW` process solves the single-phase flow equation (volume balance, no gravity in the fracture plane)

$$S \frac{\partial p}{\partial t} - \nabla \cdot \left(\frac{k}{\mu} \nabla p \right) = Q(t)\, \delta(\mathbf{x} - \mathbf{x}_{\text{well}}), \tag{1}$$

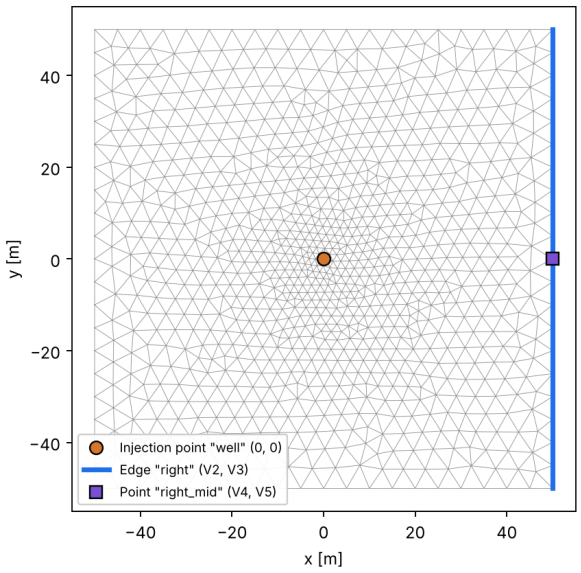


Figure 3: Mesh of the single-fracture model with injection point and the boundary geometries used.

with pressure p, specific storage S [Pa^{-1}], permeability k [m^2], viscosity $\mu = 1 \cdot 10^{-3}\,\mathrm{Pa\,s}$, density $1000\,\mathrm{kg/m}^3$, porosity 1 and injection rate Q [m^3/s] applied as a nodal source term (`pressure_source_term`). The hydraulic diffusivity $D = k/(\mu S)$ controls how fast the pressure spreads; $\sqrt{4Dt}$ is a measure of the radius of influence after time t.

3.3 Initial and boundary conditions

- V1: $p_0 = 0$, no boundary condition (closed fracture edges).
- V2, V3: $p = 1\,\mathrm{MPa}$ on the right edge ($x = 50\,\mathrm{m}$), initial pressure 1 MPa; the comparison uses $p - 1\,\mathrm{MPa}$.
- V4, V5: $p = 1\,\mathrm{MPa}$ only at the midpoint of the right edge, "right_mid" (50 m, 0), initial pressure 1 MPa.

3.4 Discretisation and solver

Backward Euler with fixed time steps, $\Delta t = 10\,\mathrm{s}$ for 51.6 (2500 s to 4000 s, 150 steps) and 5 s for 40.6 (1750 s to 2670 s, 184 steps). Picard iteration (absolute tolerance $1 \cdot 10^{-6}$), Eigen CG with diagonal preconditioner. One run takes 1–2 s. For very small storage the absolute Picard tolerance can fail (high pressures); a relative tolerance is then preferable.

4 Calibration method

4.1 Objective function

The objective is the root-mean-square error between simulated and measured pressure build-up at the injection point,

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{n=1}^{N}\left[\Delta p_{\mathrm{sim}}(t_n) - \Delta p_{\mathrm{meas}}(t_n)\right]^2}, \tag{2}$$

evaluated at the N output times of the simulation (measurement linearly interpolated), over 2500 s to 4000 s for 51.6 and 1750 s to 2670 s for 40.6 (40.6 includes the pressure decay after shut-in).

4.2 Model variants

V1 constant k, S; closed boundaries (`stimtec.prj`).

V2 constant k, S; Dirichlet condition on the right edge (`stimtec_bc_right.prj`).

V3 $k(t)$, $S(t)$ coupled to the injection rate; Dirichlet on the right edge (`stimtec_bc_right_kS_t.prj`).

V4 as V3, Dirichlet only at the edge midpoint (`stimtec_bc_rightmid_kS_t.prj`).

V5 k and S computed in Python before each time step and written into OpenGeoSys through its Python interface (`stimtec_bc_rightmid_py.prj`, `ogs_py_kS.py`): V5a reproduces V4; V5b uses a pressure-dependent law.

In V3 and V4 permeability and storage are OpenGeoSys `Function` parameters with the same time windows as the source term, mapped linearly onto the injection rate:

$$X(t) = X_{\min} + (X_{\max} - X_{\min})\,\frac{q(t) - q_{\min}}{q_{\max} - q_{\min}}, \qquad X \in \{k, S\}, \tag{3}$$

so that four parameters $k_{\min}, k_{\max}, S_{\min}, S_{\max}$ are calibrated. In V5b each element receives

$$k = \min\Big(k_0\, e^{\beta_k \Delta p},\, k_{\text{cap}}\Big), \qquad S = \min\Big(S_0\, e^{\beta_S \Delta p},\, S_{\text{cap}}\Big), \tag{4}$$

with the element pressure build-up Δp of the previous time step (explicit coupling), $k_{\text{cap}} = 1 \cdot 10^{-12}\,\text{m}^2$ and $S_{\text{cap}} = 1 \cdot 10^{-9}\,\text{Pa}^{-1}$.

4.3 Optimisation

Parameters were optimised in $\log_{10}$ space (exponents β linear): first a coarse grid (V1: $k = 10^{-16} \ldots 10^{-10}\,\text{m}^2$, $S = 10^{-12} \ldots 10^{-4}\,\text{Pa}^{-1}$, one point per decade), then Nelder–Mead. V4 was started from the V3 optimum; V5b from estimates based on V4. Each objective evaluation is a full OpenGeoSys run; V3 needed about 450, V4 about 150 and V5b about 150 runs.

5 Results: test 51.6

Table 3 and Fig. 4 summarise the results.

- **V1/V2 (constant properties):** RMSE 1.69 MPa. The model accumulates pressure almost linearly with the rate and overshoots strongly at the end (9.3 MPa instead of 6.7 MPa) while underestimating the early steps. The edge boundary condition hardly changes the result. A quasi-steady branch with small storage ($k \approx 3 \cdot 10^{-14}\,\text{m}^2$, $S \le 1 \cdot 10^{-13}\,\text{Pa}^{-1}$) is also possible, but reaches only RMSE $\approx$ 2.05 MPa, since the measured pressure is not proportional to the rate.
- **V3 (rate-dependent k, S, edge BC):** RMSE 0.334 MPa, a factor of 5 better. Permeability rises by a factor of 8 with the rate.
- **V4 (point BC):** using the V3 parameters with the point boundary gives RMSE 0.717 MPa; the boundary geometry matters for this test. Re-calibrated: RMSE 0.322 MPa, with k almost unchanged and $S_{\max}$ 2.7 times larger (the point boundary relieves less pressure, so more storage is required).

- **V5a:** identical to V4 (difference $\leq 10\,\mathrm{Pa}$), which verifies the Python coupling.
- **V5b (pressure-dependent):** RMSE 0.469 MPa. The overall trend is reproduced, but the explicit coupling produces overshoots at rate jumps (e.g. $t \approx 2810\,\mathrm{s}$), and k stays high after the rate decreases (no closure/hysteresis).

Remaining deviations of V4 (Table 5): before the injection the model already shows $\approx 0.3\,\mathrm{MPa}$ because of the small base rate of the project file (0.06 l/min, digitised 0.001 l/min); the first pressure peak and the step at 2706 s to 2800 s are underestimated by 0.6 MPa to 0.7 MPa, where the model rates (0.25 and 0.16 l/min) are clearly lower than the digitised rates (0.63 and 0.53 l/min). From 3080 s onwards the mean deviation per step is within $\pm 0.2\,\mathrm{MPa}$; at the last step (9.7 l/min) the model is 0.29 MPa too high.

Table 3: Calibration results for test 51.6 (RMSE recomputed with the corrected digitisation).

Variant	BC	k [m^2]	S [Pa^{-1}]	RMSE [MPa]	$\Delta p_{\max}$ [MPa]
V1	closed	$1.41 \cdot 10^{-14}$	$1.42 \cdot 10^{-11}$	1.688	9.34
V2	edge	$1.47 \cdot 10^{-14}$	$1.00 \cdot 10^{-11}$	1.685	9.34
V3	edge	$3.30 \cdot 10^{-15}$–$2.61 \cdot 10^{-14}$	$3.02 \cdot 10^{-14}$–$5.22 \cdot 10^{-12}$	0.334	6.76
V4	point	$3.37 \cdot 10^{-15}$–$2.41 \cdot 10^{-14}$	$2.48 \cdot 10^{-14}$–$1.41 \cdot 10^{-11}$	0.322	6.84
V5a	point	as V4	as V4	0.322	6.84
V5b[a]	point	$k_0 = 3.50 \cdot 10^{-15}$	$S_0 = 3.63 \cdot 10^{-14}$	0.469	6.88
Measured					6.75

[a] Eq. (4) with $\beta_k = 0.49\,\mathrm{MPa}^{-1}$, $\beta_S = 1.04\,\mathrm{MPa}^{-1}$.

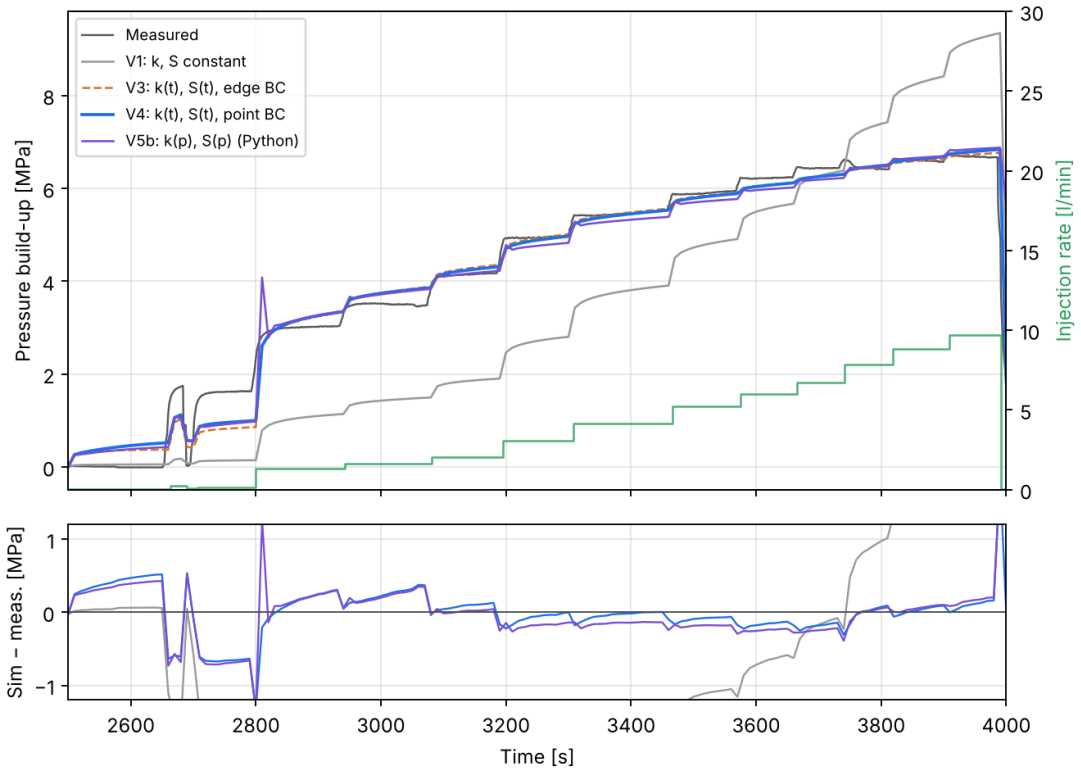


Figure 4: Test 51.6: measured pressure build-up and model variants V1, V3, V4 and V5b (V2 is practically identical to V1, V5a to V4). Bottom: residuals of V1, V4 and V5b.

6 Results: test 40.6

The same workflow was applied to test 40.6 (Table 4, Fig. 5).

- **V1/V2:** RMSE 1.163 MPa; identical results with and without boundary condition. As for 51.6 the model underestimates the low and overestimates the high rate steps (5.7 MPa instead of 3.8 MPa).
- **V3/V4:** RMSE 0.0975 MPa, a factor of 12 better than V1; V3 and V4 give the same result. Permeability rises by a factor of 28 with the rate, storage only by a factor of 4.
- **V5a:** identical to V4. **V5b:** RMSE 0.211 MPa, good overall trend but peaks at every rate jump due to the explicit coupling.

Remaining deviations of V4 are below 0.06 MPa per step except at 4 l/min (−0.12 MPa) and at 8 l/min (0.11 MPa), where the measured pressure stagnates at ≈ 3.75 MPa (Table 5). This indicates additional opening or leakage at the highest rate that the linear rate mapping does not capture.

Table 4: Calibration results for test 40.6.

Variant	BC	k [m^2]	S [Pa^{-1}]	RMSE [MPa]	$\Delta p_{\max}$ [MPa]
V1	closed	$9.15 \cdot 10^{-15}$	$1.08 \cdot 10^{-9}$	1.1631	5.69
V2	edge	$9.15 \cdot 10^{-15}$	$1.08 \cdot 10^{-9}$	1.1631	5.69
V3	edge	$6.16 \cdot 10^{-16}$–$1.73 \cdot 10^{-14}$	$2.01 \cdot 10^{-10}$–$7.95 \cdot 10^{-10}$	0.0975	3.94
V4	point	$6.16 \cdot 10^{-16}$–$1.73 \cdot 10^{-14}$	$2.01 \cdot 10^{-10}$–$7.98 \cdot 10^{-10}$	0.0975	3.94
V5a	point	as V4	as V4	0.0975	3.94
V5b[a]	point	$k_0 = 3.68 \cdot 10^{-15}$	$S_0 = 4.73 \cdot 10^{-14}$	0.2106	4.05
Measured					3.83

[a] Eq. (4) with $\beta_k = 1.06\,\text{MPa}^{-1}$, $\beta_S = 0.79\,\text{MPa}^{-1}$.

Table 5: Mean residual of V4 (simulated − measured) per injection step; rates of the model.

Test 51.6			Test 40.6		
until t [s]	q [l/min]	residual [MPa]	until t [s]	q [l/min]	residual [MPa]
2663.9	0.062	+0.33	1785.0	0.003	−0.01
2689.3	0.252	−0.60	1900.5	0.548	+0.06
2706.5	0.063	+0.23	2071.5	1.118	+0.02
2800.1	0.163	−0.72	2151.8	2.066	−0.02
2942.2	1.322	+0.12	2322.8	4.061	−0.12
3081.1	1.643	+0.23	2422.3	6.086	+0.01
3195.5	2.066	+0.06	2510.3	7.994	+0.11
3308.3	3.070	−0.08	2670.0	0.011	+0.01
3466.0	4.168	−0.05			
3575.3	5.227	−0.10			
3665.6	6.000	−0.17			
3740.8	6.716	−0.20			
3819.2	7.873	+0.01			
3908.8	8.813	+0.03			
3991.4	9.694	+0.29			

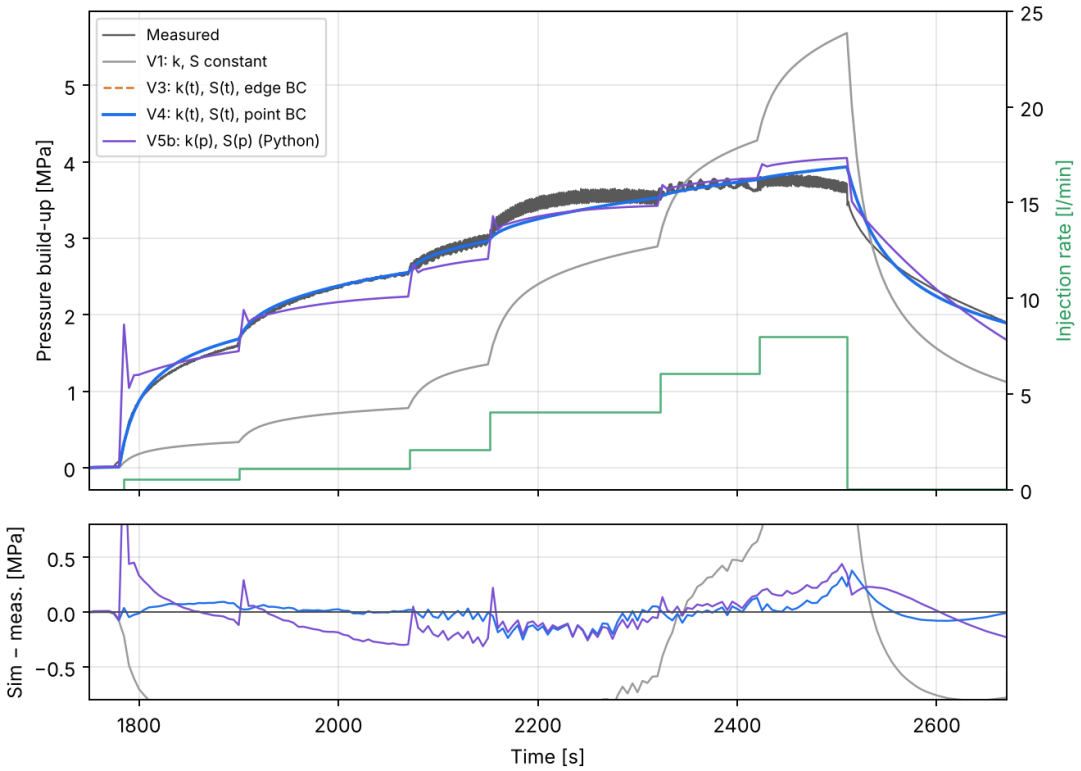


Figure 5: Test 40.6: measured pressure build-up and model variants V1, V3, V4 and V5b (V2 = V1, V3 ≈ V4, V5a = V4). Bottom: residuals of V1, V4 and V5b.

7 Comparison and discussion

7.1 Calibrated parameters

Figure 6 shows the calibrated V4 permeability and storage as functions of time, Table 6 compares the two tests.

Rate-dependent transmissivity. In both tests constant hydraulic properties fail, and permeability that increases with the injection rate is the key to a good fit. The increase is moderate in 51.6 (factor 7) and strong in 40.6 (factor 28), whereas the maximum permeability is similar ($\approx 2 \cdot 10^{-14}\,\mathrm{m}^2$). This is consistent with hydraulic opening of the fracture under increasing fluid pressure, which in 40.6 starts from a tighter initial state. The coupling to the injection rate (Eq. 3) is an empirical proxy; it requires knowledge of the rate history and cannot be transferred to other tests without re-calibration.

Storage and diffusivity. The tests differ most in storage: in 40.6, S is 15–50 times larger than at the high-rate steps of 51.6 and depends only weakly on the rate (factor 4), whereas in 51.6 it spans more than two orders of magnitude and is less well constrained. Correspondingly the hydraulic diffusivity in 51.6 is of the order of 2–140 m^2/s: the pressure reaches quasi-steady state within each step and spreads over the entire model domain. In 40.6 it is only $\approx 0.02\,\mathrm{m}^2/\mathrm{s}$: the pressure build-up within each step is transient and the decay after shut-in is slow. Figure 7 shows the computed pressure profiles at the end of the highest injection step: in 51.6 the pressure build-up is still 1.4 MPa at 50 m, in 40.6 it drops below 0.015 MPa at 10 m and below 1 Pa at 20 m.

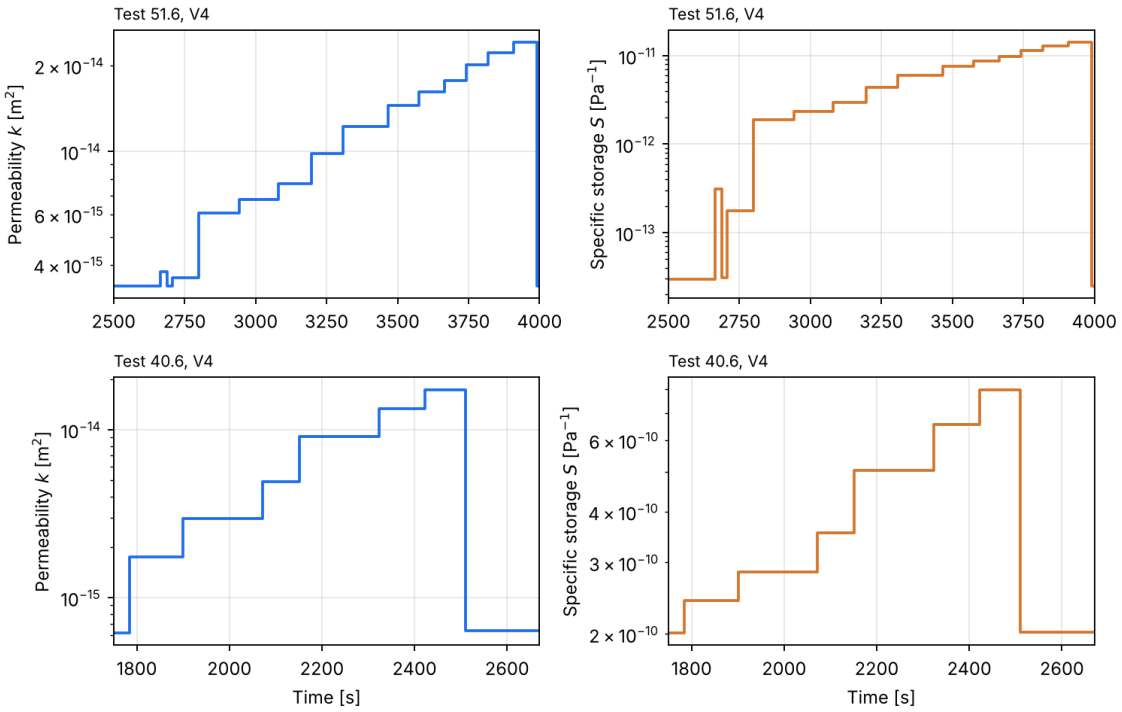


Figure 6: Calibrated permeability and specific storage of V4 (Eq. 3) for tests 51.6 (top) and 40.6 (bottom).

Table 6: Comparison of tests 51.6 and 40.6.

Quantity	Unit	51.6	40.6
Injection steps (main)	–	13	6
Max. injection rate	l/min	10.0	8.0
Max. pressure build-up	MPa	6.75	3.83
Exponent of $\Delta p \propto q^n$	–	0.47	0.31
V1: k	m^2	$1.41 \cdot 10^{-14}$	$9.15 \cdot 10^{-15}$
V1: S	Pa^{-1}	$1.42 \cdot 10^{-11}$	$1.08 \cdot 10^{-9}$
V1: RMSE	MPa	1.688	1.163
V4: $k_{\min}$ / $k_{\max}$	m^2	$3.37 \cdot 10^{-15}$ / $2.41 \cdot 10^{-14}$	$6.16 \cdot 10^{-16}$ / $1.73 \cdot 10^{-14}$
V4: $k_{\max}/k_{\min}$	–	7.2	28.1
V4: $S_{\min}$ / $S_{\max}$	Pa^{-1}	$2.48 \cdot 10^{-14}$ / $1.41 \cdot 10^{-11}$	$2.01 \cdot 10^{-10}$ / $7.98 \cdot 10^{-10}$
V4: $D = k/(\mu S)$ (min. / max. rate)	m^2/s	136 / 1.7	0.003 / 0.022
V4: RMSE	MPa	0.322	0.098
V5b: β_k / β_S	MPa^{-1}	0.49 / 1.04	1.06 / 0.79
V5b: RMSE	MPa	0.469	0.211

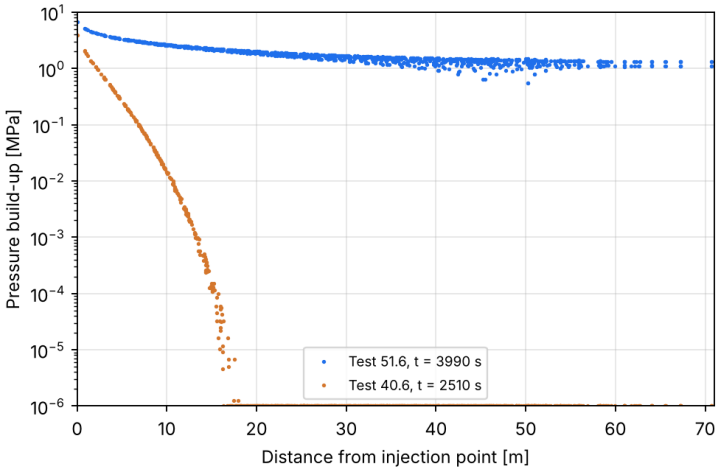


Figure 7: Pressure build-up at all mesh nodes versus distance from the injection point at the end of the highest injection step (V4).

Influence of the boundary condition. This explains the different role of the boundary condition: in 51.6 the boundary geometry is relevant (V3 parameters with the point boundary: RMSE 0.72 MPa instead of 0.33 MPa), and the calibrated storage partly compensates the model size and boundary assumptions. In 40.6 the boundary has no influence (V1 = V2, V3 = V4). For 51.6 the parameters are therefore conditional on the 100 m domain and the boundary condition; for 40.6 they are local properties of the near-well region ($\lesssim 10$ m).

Pressure-dependent formulation (V5b). The physically motivated law of Eq. (4) captures the overall behaviour with four parameters but is worse than V4 (51.6: 0.47 vs. 0.32 MPa; 40.6: 0.21 vs. 0.10 MPa). The main reasons are the explicit coupling (permeability lags one time step behind the pressure, causing spikes after rate jumps) and the missing closure or hysteresis after rate reductions. Implicit coupling (Picard iteration on $k(p^{n+1})$), smaller time steps, a closure law, or a combination $k(q, p)$ are possible improvements.

Data and model uncertainties. The evaluation relies on digitised data. For 51.6 the source term of the project file deviates from the digitised rate in the first steps (Table 1), which explains most of the remaining misfit before 2800 s; a re-calibration with the digitised steps is recommended. The single-fracture model represents the fracture as a homogeneous 1 m thick layer with a single injection point; only the products of permeability or storage and thickness are identifiable. Rock-matrix leakage and fracture-network effects are not represented.

7.2 Verification

For this report, the V4 models of both tests were rerun with OpenGeoSys 6.5.9; the computed pressures at the well agree with the stored calibration results to within 10^{-9} Pa. All RMSE values were recomputed against the current digitisation; for 51.6 the correction of 2 October 2026 changes them by less than 1 kPa. Note that the RMSE here is evaluated at the simulation output times. Evaluated at all 2852 measured samples of 40.6 the V4 value is 0.104 MPa instead of 0.098 MPa.

8 Conclusions and recommendations

1. Both injection tests show a strongly sub-linear pressure–rate relation ($\Delta p \propto q^{0.47}$ and $q^{0.31}$) that cannot be reproduced with constant permeability and storage (RMSE 1.69 and

1.16 MPa).

2. Permeability and storage increasing with the injection rate reproduce the measurements well: RMSE 0.32 MPa (51.6) and 0.098 MPa (40.6). Best variant: V4 (`stimtec_bc_rightmid_kS_t.prj`).
3. Calibrated permeability: $3.4 \cdot 10^{-15}$–$2.4 \cdot 10^{-14}\,\mathrm{m}^2$ (51.6, factor 7) and $6.2 \cdot 10^{-16}$–$1.7 \cdot 10^{-14}\,\mathrm{m}^2$ (40.6, factor 28); storage $2.5{\cdot}10^{-14}$–$1.4{\cdot}10^{-11}\,\mathrm{Pa}^{-1}$ and $2.0{\cdot}10^{-10}$–$8.0{\cdot}10^{-10}\,\mathrm{Pa}^{-1}$. The rate-dependent permeability indicates hydraulic opening of the fracture.
4. The two tests differ mainly in storage and diffusivity, so the model boundary matters for 51.6 but not for 40.6.
5. The OpenGeoSys Python interface allows arbitrary k, S laws to be prescribed per time step (verified against V4); a pressure-dependent exponential law with explicit coupling reaches RMSE 0.47 and 0.21 MPa.

Recommendations:

- Re-calibrate 51.6 with the digitised injection steps (`injektion_stufen_51.csv`).
- Develop a mechanistic, implicitly coupled pressure-dependent permeability law (with closure) that is transferable between tests, and calibrate it jointly on both tests.
- Use structural models (DFN, 3D with rock matrix) to check whether the near-well behaviour of 40.6 and the far-reaching response of 51.6 can be explained by one consistent fracture system. A DFN evaluation of 40.6 is reported separately.

A Files and workflow

Both test folders (`BH10_20180716_51.6`, `BH10_20180716_40.6`) have the same structure: `simulation/` (project files, mesh, geometry) and `workflow/` (data, scripts, results).

File	Content
`simulation/stimtec.prj`	V1 (optimum entered)
`simulation/stimtec_bc_right.prj`	V2
`simulation/stimtec_bc_right_kS_t.prj`	V3
`simulation/stimtec_bc_rightmid_kS_t.prj`	V4 (best variant)
`simulation/stimtec_bc_rightmid_py.prj`, `stimtec-square-py.vtu`	V5 (Python interface)
`simulation/stimtec-square.vtu/.gml`	Mesh and geometry (well, right, right_mid)
`workflow/digitize_*.py`	Digitisation of pressure and rate
`workflow/injektion_stufen_*.py/.csv`	Injection steps from the digitised rate
`workflow/calibrate_k_S.py`	V1, V2 (grid + Nelder–Mead)
`workflow/calibrate_kS_t.py`	V3, V4
`workflow/ogs_py_kS.py`, `calibrate_py_pressure.py`	V5a, V5b
`workflow/calibrate_*_best.csv`, `*_log.csv`	Best-fit time series and optimisation logs
`workflow/bh40.py`, `make_prj_40.py`	40.6: common definitions, project generation from 51.6

The figures of this report were produced with `make_figs_sf.py`. Typical calls (in `workflow`):

```
python calibrate_k_S.py                                          # V1
PRJ_NAME=stimtec_bc_right.prj P_REF=1e6 python calibrate_k_S.py   # V2
python calibrate_kS_t.py                                         # V3
PRJ_NAME=stimtec_bc_rightmid_kS_t.prj BC_GEOMETRY=right_mid \
    X0="..." python calibrate_kS_t.py                            # V4
py -3.13 ogs_py_kS.py                                            # V5a
py -3.13 calibrate_py_pressure.py                                # V5b
```

6 Exercise 3: STIMTEC: Hydraulic Injection Tests - Discrete Fracture Network Model

Evaluation of the STIMTEC Hydraulic Injection Test BH10_20180718_40.6

Discrete Fracture Network Model, Borehole–Fracture Coupling and Calibration of Permeability, Storage and Flow Partitioning

Olaf Kolditz*
Helmholtz Centre for Environmental Research – UFZ, Leipzig

3 October 2026

Abstract

The stepwise hydraulic injection test BH10_20180718_40.6 of the STIMTEC experiment was evaluated with a three-dimensional discrete fracture network (DFN) model in OpenGeoSys 6.5.9 (`LIQUID_FLOW`). The borehole BH10 intersects two of the four fractures of the network; the intersections were determined by an exact line–triangle intersection and coincide with mesh nodes 270 (fracture 1) and 271 (fracture 2). The injection was applied as nodal mass source terms at these two nodes. Using the injection steps of test 40.6 and constant permeability and storage (time-weighted means of the 2D reference calibration) the pressure build-up is reproduced only poorly (RMSE 1.24 MPa). Coupling permeability $k(t)$ and specific storage $S(t)$ linearly to the injection rate (variant V4) reduces the error to 0.099 MPa for the mean pressure of both intersections, but the two intersections then differ by up to 0.5 MPa. Calibrating additionally the partitioning of the injection rate between the two fractures, with an objective function that requires both intersections to match the measured interval pressure, yields a physically consistent solution with $\alpha_1 = 46.5\,\%$ of the rate entering fracture 1, $k = 3.40 \cdot 10^{-16} \ldots 9.06 \cdot 10^{-15}\,\mathrm{m}^2$, $S = 2.50 \cdot 10^{-10} \ldots 9.15 \cdot 10^{-10}\,\mathrm{Pa}^{-1}$ and an RMSE of 0.118 MPa over both intersections (0.102 MPa for their mean). The calibrated parameters are very close to those of the 2D reference model. The rate-dependent permeability (increase by a factor of ≈ 27) confirms hydraulic opening of the fractures during injection.

*This report, which is based on a systematic OGS study of the STIMTEC experiment, was supported by AI (Claude-Opus 5.5)

Contents

1 Introduction **3**

2 Field test and data **3**
2.1 Test BH10_20180718_40.6 3

3 Reference evaluation with a 2D model **4**

4 DFN model **4**
4.1 Fracture network geometry 4
4.2 Borehole–fracture intersections 5
4.3 Consistency of the supplied sub-meshes 5

5 Numerical model **6**
5.1 Governing equation 6
5.2 Initial and boundary conditions 6
5.3 Discretisation and solver 6
5.4 Aperture 6

6 Calibration method **7**
6.1 Parameterisation 7
6.2 Objective functions 7
6.3 Optimisation 7

7 Results **7**
7.1 Constant permeability and storage 7
7.2 Rate-dependent permeability and storage (V4) 8
7.3 Calibration of the rate partitioning 8

8 Discussion **11**

9 Conclusions and outlook **11**

A Files and workflow **12**

1 Introduction

The STIMTEC experiment investigates hydraulic stimulation of crystalline rock at the decametre scale. Stepwise injection tests in borehole BH10 provide pressure and flow-rate records from which the hydraulic properties of the fractured rock mass can be derived. In previous work (Sect. 3) the injection tests BH10_20180716_51.6 and BH10_20180718_40.6 were evaluated with a two-dimensional, homogeneous OpenGeoSys model with a point source at the well. That analysis showed that constant hydraulic properties cannot reproduce the measured pressure response, whereas permeability and storage that increase with the injection rate reproduce it very well.

The present report extends the evaluation of test 40.6 to a structurally more realistic model: a discrete fracture network (DFN) of four planar fractures in which borehole BH10 intersects two fractures. The objectives are

1. to determine the mesh nodes at which BH10 intersects the fracture network, so that the injection can be prescribed there;
2. to set up a consistent OpenGeoSys project with the injection steps of test 40.6;
3. to calibrate permeability k and specific storage S against the measured pressure build-up, first with constant and then with rate-dependent properties; and
4. to calibrate the partitioning of the injection rate between the two intersected fractures.

The main quantity of interest is the fluid pressure in the injection interval; the main parameters are permeability and storage.

2 Field test and data

2.1 Test BH10_20180718_40.6

The test consists of six injection steps between 0.55 and 8.0 l/min followed by shut-in (Table 1, Fig. 1). The data are stored in the project folder `BH10_20180716_40.6` (the source graph is labelled 20180718; both names refer to the same test).

The pressure ("P int d") and the downhole flow rate were digitised directly from the vector paths of the source PDF (`digitize_bh10_40.py`), with axes calibrated via the tick marks, giving 2852 pressure samples between 1750 s and 2670 s. The injection steps were derived from the jumps in the smoothed rate; the value of each step is the mean of the raw rate in that window (`injektion_stufen_40.py`). The baseline pressure (median for $t < 1780$ s) is 0.087 MPa; all comparisons in this report refer to the pressure build-up $\Delta p = p - p_{\mathrm{base}}$. The maximum measured build-up is 3.83 MPa.

Table 1: Injection steps of test 40.6 (`injektion_stufen_40.csv`). Step i applies until t_{end}.

Step	t_{end} [s]	q [l/min]	q [m^3/s]	$\dot{m}$ [kg/s]
0 (pre-injection)	1785.0	0.003	$4.55 \cdot 10^{-8}$	$4.55 \cdot 10^{-5}$
1	1900.5	0.548	$9.13 \cdot 10^{-6}$	$9.13 \cdot 10^{-3}$
2	2071.5	1.118	$1.86 \cdot 10^{-5}$	$1.86 \cdot 10^{-2}$
3	2151.8	2.066	$3.44 \cdot 10^{-5}$	$3.44 \cdot 10^{-2}$
4	2322.8	4.061	$6.77 \cdot 10^{-5}$	$6.77 \cdot 10^{-2}$
5	2422.3	6.086	$1.01 \cdot 10^{-4}$	$1.01 \cdot 10^{-1}$
6	2510.3	7.994	$1.33 \cdot 10^{-4}$	$1.33 \cdot 10^{-1}$
shut-in	2669.8	0.011	$1.86 \cdot 10^{-7}$	$1.86 \cdot 10^{-4}$

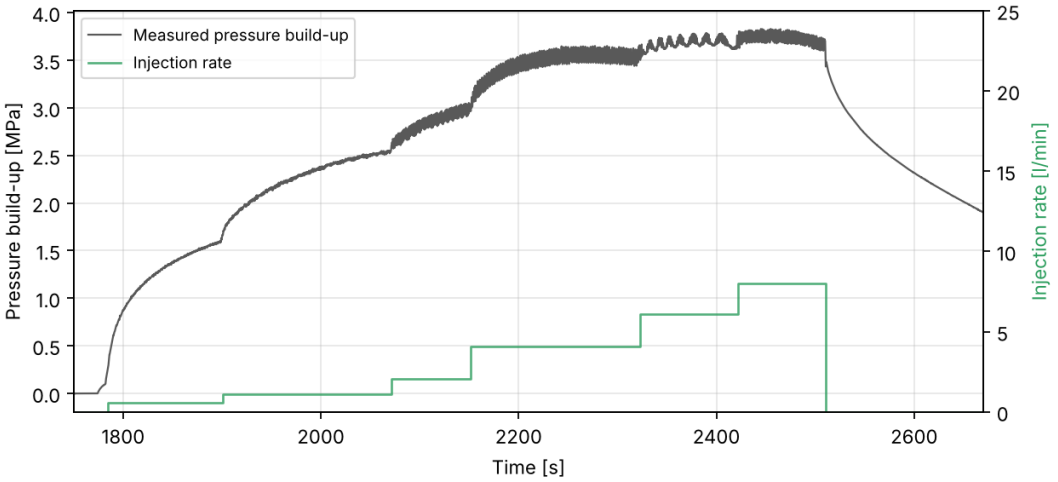


Figure 1: Measured pressure build-up (baseline 0.087 MPa removed) and injection steps of test 40.6. The pressure increases quickly at each rate step and then flattens; at 8 l/min it stagnates at ≈ 3.75 MPa. The rate increases by a factor of 14.6 between the first and last step, the pressure only by a factor of ≈ 2.3.

3 Reference evaluation with a 2D model

Before setting up the DFN model, test 40.6 was evaluated with a 2D OpenGeoSys `LIQUID_FLOW` model (100 m × 100 m, point source "well" at the centre, unit thickness, $\Delta t = 5$ s). The objective function was the RMSE of the pressure build-up at the well over 1750 s to 2670 s, minimised by a log-grid search followed by Nelder–Mead in $\log_{10}$ space. Table 2 summarises the variants.

Table 2: 2D reference calibration of test 40.6.

Variant	Boundary	k [m^2]	S [Pa^{-1}]	RMSE [MPa]
V1: k, S constant	closed	$9.15 \cdot 10^{-15}$	$1.08 \cdot 10^{-9}$	1.163
V2: k, S constant	edge $p = 1$ MPa	$9.15 \cdot 10^{-15}$	$1.08 \cdot 10^{-9}$	1.163
V3: $k(t)$, $S(t)$	edge	$6.16 \cdot 10^{-16}$–$1.73 \cdot 10^{-14}$	$2.01 \cdot 10^{-10}$–$7.95 \cdot 10^{-10}$	0.097
V4: $k(t)$, $S(t)$	point $p = 1$ MPa	$6.16 \cdot 10^{-16}$–$1.73 \cdot 10^{-14}$	$2.01 \cdot 10^{-10}$–$7.98 \cdot 10^{-10}$	0.097
V5b: $k(p)$, $S(p)$[a]	point	$k_0 = 3.68 \cdot 10^{-15}$	$S_0 = 4.73 \cdot 10^{-14}$	0.211

[a] $k = k_0 \exp(\beta_k \Delta p)$, $S = S_0 \exp(\beta_S \Delta p)$ per element, $\beta_k = 1.06$ MPa^{-1}, $\beta_S = 0.79$ MPa^{-1}, explicit coupling via the OpenGeoSys Python interface.

The key findings of the 2D evaluation are: (i) constant properties lead to a large structural error (RMSE ≈ 1.16 MPa); (ii) permeability that increases with the injection rate (factor ≈ 28) and moderately rate-dependent storage reproduce the measurement with an RMSE of 0.10 MPa; (iii) the boundary condition is irrelevant, because the hydraulic diffusivity is small ($D = k/(\mu S) \approx 0.02$ m^2/s) and the pressure front reaches only ≈ 10 m before shut-in. The DFN evaluation below uses variant V4 as reference.

4 DFN model

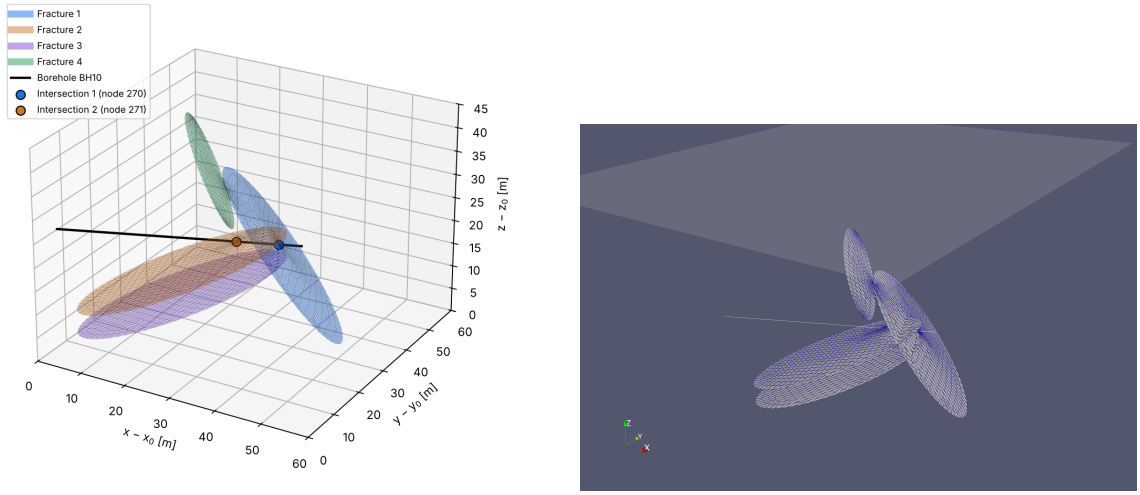
4.1 Fracture network geometry

The DFN mesh `mesh.vtu` consists of 1834 nodes and 3440 linear triangles (mean edge length 1.8 m) representing four planar, approximately elliptical fractures embedded in 3D (MaterialIDs 1–4; Table 3, Fig. 2). Fracture 1 intersects all other fractures (10, 7 and 4 shared nodes with fractures 2, 3 and 4), whereas fractures 2, 3 and 4 do not intersect each other. Fractures 2 and 3

are parallel, gently dipping planes whose centres are offset vertically by 4.6 m; fractures 1 and 4 dip steeply in the same direction. Coordinates are given in Gauss–Krüger metres (origin of the plots: minimum coordinate of the mesh).

Table 3: Fractures of the DFN model (orientation from a plane fit through the nodes).

Fracture	Triangles	Nodes	Area [m^2]	Dip dir./dip [°]	Axes [m]	Intersected by BH10
1	1076	572	1599	053/53	53×38	yes (node 270)
2	1017	542	1434	270/24	44×41	yes (node 271)
3	882	475	1434	270/24	45×41	no (1.3 m off)
4	465	266	497	053/55	51×13	no (> 10 m)



(a) Fractures, BH10 and the two intersections.

(b) DFN mesh and BH10 (ParaView).

Figure 2: Geometry of the DFN model and borehole BH10.

4.2 Borehole–fracture intersections

Borehole BH10 (`BH10.vtu`) is a straight line of length 55.61 m with direction $(-0.497, -0.820, 0.283)$, discretised by 21 points. The intersections with the fracture network were determined by intersecting the borehole axis with all 3440 triangles using the Möller–Trumbore algorithm (script `find_bh10_intersections.py`). The borehole intersects exactly two fractures (Table 4). Both intersection points coincide with existing mesh nodes (distance $< 1 \cdot 10^{-4}$ m) and with points 2 and 6 of `BH10.vtu`; they are 10.05 m apart. The plane of fracture 3 is crossed at 5.65 m but outside the fracture area (minimum distance of the axis to the fracture 1.28 m); fracture 4 is at least 10.7 m away from the axis.

Table 4: Intersections of BH10 with the fracture network. s: distance along BH10 from its deep end.

Fracture	Mesh node	x [m]	y [m]	z [m]	s [m]
1	270	4 595 094.5722	5 644 196.6047	268.9331	5.42
2	271	4 595 089.5745	5 644 188.3609	271.7744	15.47

4.3 Consistency of the supplied sub-meshes

The original project file `stimtec_dfn.prj` prescribed the injection as a Neumann condition on `BH10.vtu` and a hydrostatic Dirichlet condition on `bc_zmax.vtu`. Neither sub-mesh belongs to

`mesh.vtu`: the `bulk_node_ids` of `BH10.vtu` (404–535) point to nodes 8 m to 53 m away, 19 of its 21 points are not mesh nodes, and `bc_zmax.vtu` lies at $z = 313.65$ m, above the top of the DFN ($z_{\max} = 293.85$ m), with `bulk_element_ids` up to 34925. Both were evidently extracted from a 3D matrix mesh, which will be addressed in a later study. For the DFN model new point sub-meshes with correct `bulk_node_ids` were created (`BH10_frac1.vtu`, `BH10_frac2.vtu`, `BH10_intersections.vtu`), and `bc_zmax.vtu` was removed.

5 Numerical model

5.1 Governing equation

Flow in each fracture is described by the OpenGeoSys `LIQUID_FLOW` process on lower-dimensional (2D) elements in 3D space, in mass-balance form, with the fracture aperture a as a thickness factor:

$$a\,\rho\,S\,\frac{\partial p}{\partial t} - \nabla \cdot \left[a\,\rho\,\frac{k}{\mu}\,(\nabla p - \rho\,\mathbf{g}) \right] = \dot{m}, \tag{1}$$

with pressure p, specific storage S, permeability k, fluid density $\rho = 1000\,\mathrm{kg/m^3}$, viscosity $\mu = 1 \cdot 10^{-3}\,\mathrm{Pa\,s}$ and gravity $\mathbf{g} = (0, 0, -9.8)\,\mathrm{m/s^2}$. The injection enters as nodal mass sources $\dot{m}_i$ [kg/s] at the intersection nodes. Since both the storage and the conductance terms scale with a, the pressure response to a given injection rate scales with $1/a$; only the products $a\,k$ (transmissivity-type) and $a\,S$ (storativity-type) can be identified from the test.

5.2 Initial and boundary conditions

- **Initial condition:** hydrostatic pressure $p_0 = 1 \cdot 10^5\,\mathrm{Pa} + \rho g\,(z - 313.648\,\mathrm{m})$.
- **Injection:** nodal source terms `q_frac1` at node 270 and `q_frac2` at node 271, $\dot{m}_1 = \alpha_1\,\rho\,q(t)$, $\dot{m}_2 = (1 - \alpha_1)\,\rho\,q(t)$ with $q(t)$ from Table 1. In the original project the rate parameter `q_in` was given per metre of borehole (kg/(s m), divided by 55.6 m); it was converted to a total mass rate.
- **Fracture rims:** Dirichlet $p = p_0$ on all 262 rim nodes of the network (`bc_rim.vtu`). Without any Dirichlet condition and with the zero storage of the original project the problem is ill-posed (incompressible closed system; computed pressures $\sim 1 \cdot 10^{15}$ MPa). A test with closed rims and calibrated parameters (Sect. 8) changed the pressure at the intersections by less than 210 Pa, i.e. the rim condition only serves to make the problem well-posed.

5.3 Discretisation and solver

Simulation period 1750 s to 2670 s with a fixed time step $\Delta t = 5$ s (184 steps, as in the 2D reference), backward Euler, Picard iteration (relative tolerance $1 \cdot 10^{-10}$) and the Eigen SparseLU direct solver. A run takes ≈ 6 s (output on the intersection points only) to 10 s (full output). The pressure build-up at the intersections is evaluated relative to the initial (hydrostatic) pressure.

5.4 Aperture

The original project uses an aperture of $a = 1\,\mu\mathrm{m}$ for all fractures. The 2D reference model, however, implicitly uses a unit thickness of 1 m, so its k and S are per metre of thickness. To transfer the 2D results consistently, the DFN calculations below use $a = 1$ m; the calibrated k and S are therefore equivalent values for a 1 m thick layer. For a physical aperture a the corresponding values are $k_a = k \cdot (1\,\mathrm{m}/a)$ and $S_a = S \cdot (1\,\mathrm{m}/a)$ (Eq. 1).

6 Calibration method

6.1 Parameterisation

Following variant V4 of the 2D model, permeability and specific storage are piecewise constant in time and mapped linearly onto the injection steps:

$$X(t) = X_{\min} + (X_{\max} - X_{\min})\,\frac{q(t) - q_{\min}}{q_{\max} - q_{\min}}, \qquad X \in \{k, S\}, \tag{2}$$

using the same time windows as the injection (OpenGeoSys `Function` parameters `permeability_t`, `storage_t`, applied to all four fractures). The calibration parameters are $\boldsymbol{\theta} = (\log_{10} k_{\min}, \log_{10} k_{\max}, \log_{10} S_{\min}, \log_{10} S_{\max})$ and, in the last step, the rate fraction α_1 of fracture 1, transformed as $\operatorname{logit} \alpha_1 = \ln[\alpha_1/(1-\alpha_1)]$ to keep it within $(0, 1)$.

6.2 Objective functions

With $\Delta p_1(t)$, $\Delta p_2(t)$ the simulated build-up at nodes 270 and 271 and $\Delta p_m(t)$ the measured build-up at the $N = 2852$ sample times:

$$\mathrm{RMSE}_{\mathrm{mean}} = \sqrt{\frac{1}{N}\sum_j \left(\tfrac{1}{2}\left[\Delta p_1 + \Delta p_2\right] - \Delta p_m\right)_j^2}, \tag{3}$$

$$\mathrm{RMSE}_{\mathrm{nodes}} = \sqrt{\frac{1}{2N}\sum_j \left[(\Delta p_1 - \Delta p_m)_j^2 + (\Delta p_2 - \Delta p_m)_j^2\right]}. \tag{4}$$

Both intersections lie within the same packed injection interval and must therefore experience the same pressure. $\mathrm{RMSE}_{\mathrm{nodes}}$ enforces this, whereas $\mathrm{RMSE}_{\mathrm{mean}}$ allows compensating deviations of the two intersections.

6.3 Optimisation

Nelder–Mead in the transformed parameter space (initial simplex steps 0.3–0.5 decades, ±0.6 in logit space), started from the 2D V4 optimum, restarted from the optimum until no further improvement occurred (`calibrate_dfn_kS_t.py`, `calibrate_dfn_kS_split.py`). Each objective evaluation is a full OpenGeoSys run. In total about 300 runs were needed for the four-parameter and about 330 runs for the five-parameter calibration.

7 Results

7.1 Constant permeability and storage

As a first step, the time-weighted means of the 2D V4 step values over 1750 s to 2670 s were applied to all fractures: $k = 6.12 \cdot 10^{-15}\,\mathrm{m}^2$, $S = 3.98 \cdot 10^{-10}\,\mathrm{Pa}^{-1}$, $\alpha_1 = 0.5$. With the original aperture $a = 1\,\mu\mathrm{m}$ the computed build-up reaches $\approx 6 \cdot 10^6$ MPa, which illustrates the $1/a$ scaling of Eq. (1). With $a = 1$ m the maximum build-up is 6.1 MPa (fracture 1) and 5.4 MPa (fracture 2), compared with 3.83 MPa measured (Fig. 3). The RMSE of 1.24 MPa is close to that of the 2D model with constant properties (1.16 MPa) and shows the same structural error: the pressure is underestimated at low rates and overestimated at high rates, and the simulated pressure decays too fast after shut-in.

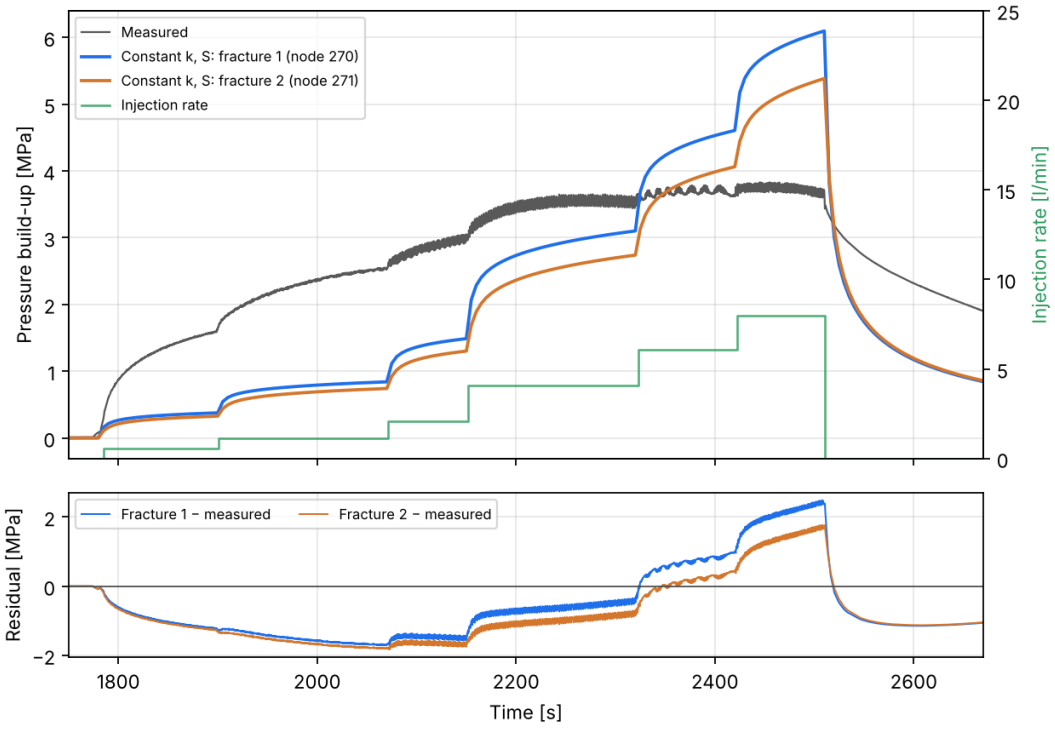


Figure 3: DFN model with constant k and S (means of 2D V4), $a = 1\,\mathrm{m}$, 50/50 rate split. $\mathrm{RMSE}_{\mathrm{mean}} = 1.24\,\mathrm{MPa}$.

7.2 Rate-dependent permeability and storage (V4)

Calibrating $k_{\min}$, $k_{\max}$, $S_{\min}$, $S_{\max}$ with a 50/50 rate split and $\mathrm{RMSE}_{\mathrm{mean}}$ as objective gives $k = 2.98 \cdot 10^{-16} \ldots 8.25 \cdot 10^{-15}\,\mathrm{m}^2$, $S = 3.14 \cdot 10^{-10} \ldots 1.77 \cdot 10^{-9}\,\mathrm{Pa}^{-1}$ and $\mathrm{RMSE}_{\mathrm{mean}} = 0.099\,\mathrm{MPa}$, practically equal to the 2D V4 result (0.098 MPa; Fig. 4). A first optimisation run ended at RMSE 0.104 MPa with $S_{\max} = 3.65 \cdot 10^{-9}$, indicating that $S_{\max}$ is weakly determined, whereas the permeability bounds are robust.

However, the two intersections behave differently: fracture 1 reaches 4.21 MPa, fracture 2 only 3.67 MPa, the RMS difference between both is 0.48 MPa, and $\mathrm{RMSE}_{\mathrm{nodes}} = 0.26\,\mathrm{MPa}$. The good fit of the mean is thus partly the result of compensating errors, which is inconsistent with a common interval pressure.

7.3 Calibration of the rate partitioning

A scan of α_1 with the V4 parameters fixed (Fig. 5) shows that $\mathrm{RMSE}_{\mathrm{mean}}$ hardly constrains the partitioning (0.099 MPa to 0.111 MPa for $\alpha_1 = 0.4 \ldots 0.6$), whereas $\mathrm{RMSE}_{\mathrm{nodes}}$ has a sharp minimum. The five-parameter calibration with $\mathrm{RMSE}_{\mathrm{nodes}}$ converged after 232 runs; a restart (100 runs) confirmed the optimum (Table 5, Fig. 6):

$$\alpha_1 = 0.465, \quad k = 3.40 \cdot 10^{-16} \ldots 9.06 \cdot 10^{-15}\,\mathrm{m}^2, \quad S = 2.50 \cdot 10^{-10} \ldots 9.15 \cdot 10^{-10}\,\mathrm{Pa}^{-1},$$

i.e. 46.5 % of the injected fluid enters fracture 1 and 53.5 % fracture 2. Both intersections now follow the measured curve (maxima 3.91 MPa and 3.98 MPa); the RMS difference between them is reduced from 0.48 MPa to 0.12 MPa, and $\mathrm{RMSE}_{\mathrm{nodes}}$ from 0.262 MPa to 0.118 MPa. $\mathrm{RMSE}_{\mathrm{mean}}$ increases only slightly (0.099 → 0.102 MPa). The calibrated permeability and storage are now very close to the 2D V4 values (Fig. 7).

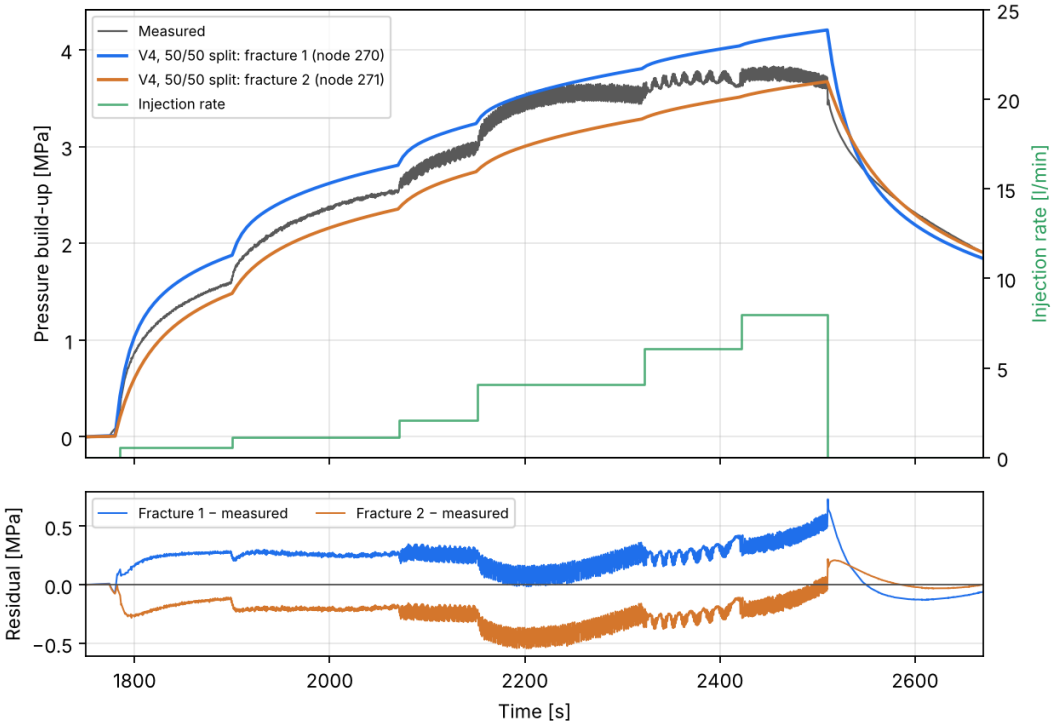


Figure 4: DFN model V4 ($k(t)$, $S(t)$ coupled to the injection rate), 50/50 rate split. $\mathrm{RMSE}_{\mathrm{mean}} = 0.099\,\mathrm{MPa}$, $\mathrm{RMSE}_{\mathrm{nodes}} = 0.262\,\mathrm{MPa}$.

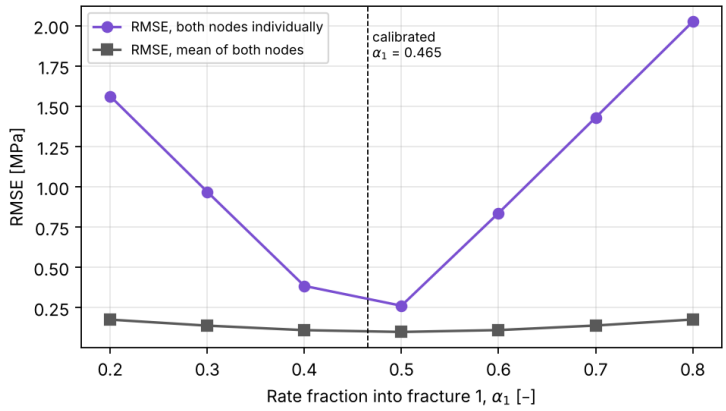


Figure 5: Sensitivity of both objective functions to the rate fraction α_1 of fracture 1 (V4 parameters with 50/50 split held fixed). Only $\mathrm{RMSE}_{\mathrm{nodes}}$ identifies the partitioning.

Table 5: Summary of the DFN calibration of test 40.6 ($a = 1\,\mathrm{m}$, Dirichlet p_0 on the fracture rims). RMS_{12}: RMS difference between the two intersections; $\Delta p_{\max}$: maximum build-up at fracture 1 / 2 (measured: 3.83 MPa). For 2D V4 the RMSE refers to the single well node.

Variant	k [m^2]	S [Pa^{-1}]	α_1	$\mathrm{RMSE}_{\mathrm{mean}}$ [MPa]	$\mathrm{RMSE}_{\mathrm{nodes}}$ [MPa]	RMS_{12} [MPa]	$\Delta p_{\max}$ [MPa]
Constant	$6.12 \cdot 10^{-15}$	$3.98 \cdot 10^{-10}$	0.5	1.242	1.256	0.37	6.09 / 5.38
V4	$2.98 \cdot 10^{-16}$–$8.25 \cdot 10^{-15}$	$3.14 \cdot 10^{-10}$–$1.77 \cdot 10^{-9}$	0.5	0.099	0.262	0.49	4.20 / 3.67
V4 + split	$3.40 \cdot 10^{-16}$–$9.06 \cdot 10^{-15}$	$2.50 \cdot 10^{-10}$–$9.15 \cdot 10^{-10}$	0.465	0.102	**0.118**	0.12	3.91 / 3.98
2D V4 (ref.)	$6.16 \cdot 10^{-16}$–$1.73 \cdot 10^{-14}$	$2.01 \cdot 10^{-10}$–$7.98 \cdot 10^{-10}$	–	0.098	–	–	–

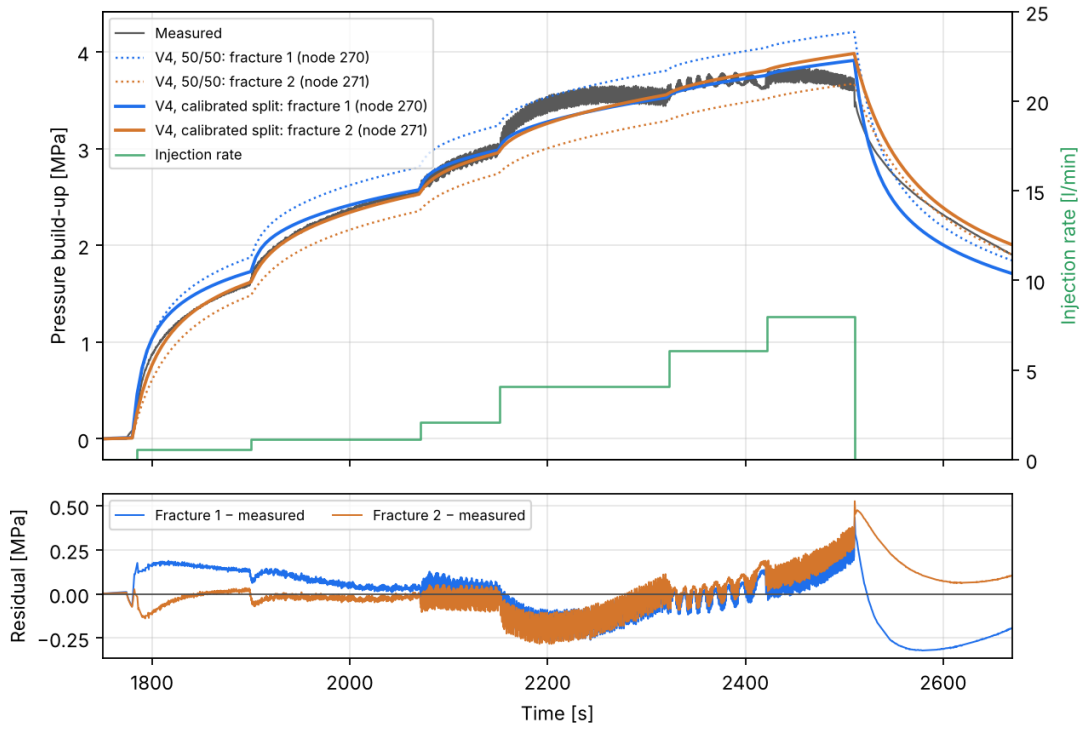


Figure 6: DFN model V4 with calibrated rate partitioning ($\alpha_1 = 0.465$; solid) compared with the 50/50 split (dotted). Residuals refer to the calibrated model. $\mathrm{RMSE}_{\mathrm{nodes}} = 0.118\,\mathrm{MPa}$, $\mathrm{RMSE}_{\mathrm{mean}} = 0.102\,\mathrm{MPa}$.

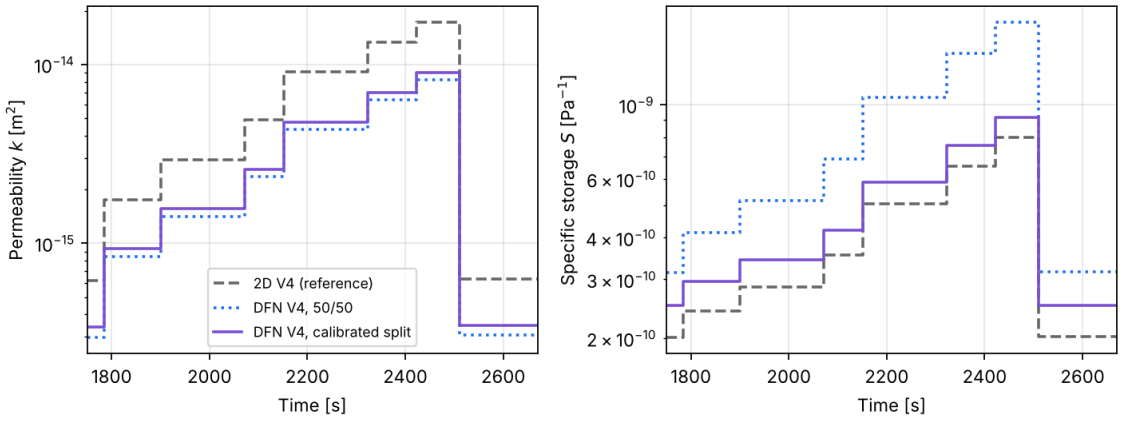


Figure 7: Calibrated permeability and specific storage as functions of time (Eq. 2) for the 2D reference V4 and the two DFN variants (values for unit thickness / $a = 1\,\mathrm{m}$).

Table 6: Mean residual (simulated − measured, MPa) per injection step for fracture 1 / fracture 2.

Step [l/min]	Constant k, S	V4 (50/50)	V4 + split
0.003 (pre)	−0.04 / −0.04	−0.01 / −0.03	0.00 / −0.02
0.548	−0.97 / −1.02	+0.25 / −0.17	+0.15 / −0.02
1.118	−1.51 / −1.61	+0.26 / −0.20	+0.07 / −0.02
2.066	−1.47 / −1.66	+0.26 / −0.24	+0.02 / −0.03
4.061	−0.64 / −1.01	+0.15 / −0.37	−0.11 / −0.11
6.086	+0.66 / +0.11	+0.28 / −0.25	0.00 / +0.04
7.994	+2.06 / +1.34	+0.41 / −0.13	+0.12 / +0.18
shut-in	−0.94 / −0.92	−0.02 / +0.03	−0.23 / +0.14

8 Discussion

Rate-dependent hydraulic properties. As in the 2D evaluation, constant hydraulic properties cannot explain the strongly sub-linear relation between injection rate and pressure. In the best DFN variant the permeability increases by a factor of 26.7 from the lowest to the highest injection step, while storage increases by a factor of 3.7. This is consistent with hydraulic opening of the fractures (pressure-dependent aperture) during injection. The coupling to the injection rate in Eq. (2) is an empirical proxy for this mechanism.

Comparison with the 2D model. The DFN model reproduces the measurement as well as the 2D model, with very similar parameters once the rate partitioning is calibrated (Fig. 7). This is expected, because the pressure perturbation remains local: with $D = k_{\max}/(\mu S_{\max}) \approx 0.01\,\mathrm{m}^2/\mathrm{s}$ the diffusion length $\sqrt{4Dt}$ over the injection period of 725 s is only $\approx 5\,\mathrm{m}$. Each intersection therefore behaves like a radial source in a quasi-infinite plane, and the fracture network geometry has little influence on the pressure in the injection interval during this test. Consistently, replacing the Dirichlet condition on the fracture rims by a no-flow condition changes the intersection pressures by less than 210 Pa, although node 270 lies only 2.8 m from the nearest rim node.

Identifiability. The permeability bounds are well constrained and reproducible across restarts. The upper storage value $S_{\max}$ is weakly constrained ($9.2 \cdot 10^{-10}$ to $3.7 \cdot 10^{-9}\,\mathrm{Pa}^{-1}$ for similar RMSE in different variants). Because of Eq. (1), k and S cannot be separated from the aperture: only $a\,k$ and $a\,S$ are identifiable. The rate partitioning α_1 is only identifiable if both intersections are required to match the interval pressure ($\mathrm{RMSE}_{\mathrm{nodes}}$).

Rate partitioning. With identical properties in all fractures a slightly smaller share of the rate enters fracture 1. A time-constant split cannot keep both intersections at exactly the same pressure: at low rates fracture 1 is too high (0.15 MPa), and after shut-in its pressure decays faster than that of fracture 2 (-0.23 vs. 0.14 MPa). In reality the borehole interval enforces a common pressure, and the split adjusts itself in time according to the transmissivity and storage of each fracture.

Remaining misfit. The largest residuals of the best variant occur at the 4 l/min step (-0.11 MPa) and at the 8 l/min step (0.12 MPa to 0.18 MPa), where the measured pressure stagnates at ≈ 3.75 MPa. The same pattern was found in the 2D model and indicates additional fracture opening or leakage at the highest rate, which the linear rate mapping does not capture.

Limitations. The evaluation relies on digitised data, a single injection test, identical properties for all fractures, a fixed time-invariant rate split, an equivalent aperture of 1 m, and an empirical (rate-based) rather than mechanistic (pressure- or stress-based) permeability model. The rock matrix is not represented.

9 Conclusions and outlook

1. Borehole BH10 intersects two fractures of the DFN, at mesh nodes 270 (fracture 1, 5.42 m from the deep end of BH10) and 271 (fracture 2, 15.47 m). The supplied borehole and top-boundary sub-meshes do not belong to the DFN mesh and were replaced by point sub-meshes with correct bulk node IDs.
2. The DFN model with nodal mass sources at the intersections, hydrostatic initial pressure and Dirichlet conditions on the fracture rims is well-posed and runs in a few seconds, which allows automated calibration.

3. Constant permeability and storage fail (RMSE 1.24 MPa). Rate-dependent properties (V4) reproduce the test with an RMSE of about 0.1 MPa.
4. A consistent solution requires both intersections to match the interval pressure. Calibrating the rate partitioning gives $\alpha_1 = 0.465$ for fracture 1, $k = 3.4 \cdot 10^{-16} \ldots 9.1 \cdot 10^{-15}\,\mathrm{m}^2$ and $S = 2.5 \cdot 10^{-10} \ldots 9.1 \cdot 10^{-10}\,\mathrm{Pa}^{-1}$ (for $a = 1\,\mathrm{m}$), with RMSE 0.118 MPa over both intersections. These values agree closely with the 2D reference evaluation.

Recommended next steps:

- Couple both intersections physically through the borehole (e.g. a highly permeable 1D borehole element or a common interval-pressure constraint) instead of a prescribed rate split, so that the partitioning can vary in time.
- Assign separate k and S to the intersected fractures and test whether the remaining differences between the intersections disappear.
- Replace the empirical rate mapping by a pressure-dependent aperture/permeability law with implicit coupling (e.g. via the OpenGeoSys Python interface with Picard iteration), and convert to physical apertures.
- Apply the same workflow to test BH10_20180716_51.6 and to the 3D model including the rock matrix (for which `bc_zmax.vtu` was prepared).

A Files and workflow

All files are in the project folder `stimtec/dfn`. The original `stimtec_dfn.prj` was not modified.

File	Content
`find_bh10_intersections.py`	Line–triangle intersection of BH10 with the DFN
`BH10_intersections.vtu/.csv`, `BH10_frac1.vtu`, `BH10_frac2.vtu`	Point sub-meshes at nodes 270/271
`bc_rim.vtu`	262 rim nodes of the fractures (Dirichlet p_0)
`make_prj_dfn_st.py`	Nodal source terms instead of Neumann on BH10 (`-rim`: rim BC)
`make_prj_dfn_40.py`	Injection steps 40.6, constant k, S, aperture option
`make_prj_dfn_40_kS_t.py`	V4 parameterisation $k(t)$, $S(t)$; options `-split1`, `-calib`
`calibrate_dfn_kS_t.py`	Calibration of k, S ($\mathrm{RMSE}_{\mathrm{mean}}$)
`calibrate_dfn_kS_split.py`	Calibration of k, S, α_1 ($\mathrm{RMSE}_{\mathrm{nodes}}$)
`*_log.csv`, `*_best.txt`	Optimisation logs and optima
`stimtec_dfn_40_a1.prj`	Constant k, S, $a = 1\,\mathrm{m}$
`stimtec_dfn_40_kS_t.prj`	V4, 50/50 split
`stimtec_dfn_40_kS_split.prj`	V4 with calibrated split (best variant)

Typical calls (in `stimtec/dfn`, with `ogs` 6.5.9 on the path):

```
python make_prj_dfn_st.py stimtec_dfn.prj stimtec_dfn_st_rim.prj --rim
python make_prj_dfn_40.py stimtec_dfn_st_rim.prj stimtec_dfn_40_a1.prj \
       --aperture=1
python calibrate_dfn_kS_t.py
X0="-15.5265,-14.0834,-9.5033,-8.7521,0.47" OBJ=nodes \
       python calibrate_dfn_kS_split.py
python make_prj_dfn_40_kS_t.py stimtec_dfn_40_a1.prj \
       stimtec_dfn_40_kS_split.prj --kmin=3.3989e-16 \
       --kmax=9.0585e-15 --smin=2.5043e-10 --smax=9.1470e-10 \
       --split1=0.4652
ogs stimtec_dfn_40_kS_split.prj -o out
```

7 Exercise 4: Selke: Catchment Model

OpenGeoSys Training Korea/China 2026

Exercise E3: Selke Catchment – A Real-World Groundwater Model
An Essay

Olaf Kolditz
Helmholtz Centre for Environmental Research – UFZ, Leipzig

7 October 2026

Abstract

The Selke exercise of the OpenGeoSys (OGS) training course 2026 is based on a Jupyter tutorial prepared for the OGS Community Meeting 2026 by Erik Nixdorf, Niklas Ritter, Tobias Meisel and Linus Walter. It builds a 2D groundwater model of the lower Selke catchment in the northern Harz foreland, Central Germany, and uses the `ogstools` Python library to set up, run and analyse it. The model solves the pressure-based groundwater flow equation of the `LiquidFlow` process with homogeneous aquifer properties, Dirichlet conditions at rivers and mining lakes and a groundwater recharge of 100 mm/a. Three scenarios are compared: constant recharge, seasonal recharge and a rise of the water level of Lake Concordia by 18 m. This essay summarises the tutorial and argues that its value lies less in the hydrogeological result than in the workflow: a real catchment, from meshes to scenario differences, is handled in a few dozen lines of Python, and every modelling decision, in particular the boundary conditions and the neglect of storage, becomes visible and can be questioned.

1 Introduction

Lectures 1 and 2 derived the groundwater flow equation from the general balance equation of continuum mechanics. Exercises E1 (Theis) and E2 (STIMTEC) applied it to a single well and to a pumping test in fractured rock. Exercise E3 takes the step to a real catchment, with irregular boundaries, rivers, lakes and pumping wells, where the result depends as much on the conceptual model as on the equation.

The motivation of the tutorial is a dramatic event. On 18 July 2009 about 2.2 million cubic metres of earth slid into the Concordia lake at Nachterstedt, a former lignite open-cast mine that was being flooded at the time. The landslide was probably triggered by soil liquefaction during the rise of the groundwater and lake level. Groundwater levels around the mining lakes are therefore not only a water-resources question but also a geotechnical one, and the Selke catchment, with its post-mining lakes and infiltration wells, has been studied in detail by Zhang et al. [4] and Nixdorf et al. [3].

This essay follows the notebook. Section 2 describes the site and the model domain, Sect. 3 the governing equation and the model setup, Sect. 4 the base case, Sect. 5 the scenarios, and Sect. 6 discusses what the exercise teaches.

2 Site and model domain

The Selke is a fourth-order stream of 64 km length that drains the forested Harz mountains and the agricultural northern Harz foreland. Earlier work built a complex 3D hydrogeological model of the catchment [4] and then reduced its stratigraphy with the open-source tools Stratmerge

and OpenGeoSys, analysing how much simplification the simulated heads can tolerate [3]. The tutorial goes one step further and uses a 2D model with a single homogeneous aquifer.

The domain (Fig. 1) covers about 40×40 km. It is bounded in the north and west by the River Bode and in the south-east by the River Wipper; the River Selke crosses it from south to north-west. Three mining lakes, Concordia, Königsauer and Wilsleber, lie in the centre, and a number of pumping wells are distributed over the basin. All these features are provided as separate meshes (`.vtu`) and loaded with `ot.Meshes.from_files`; `meshes.plot` and `model.plot_constraints` show the domain together with the assigned boundary conditions in a single figure.

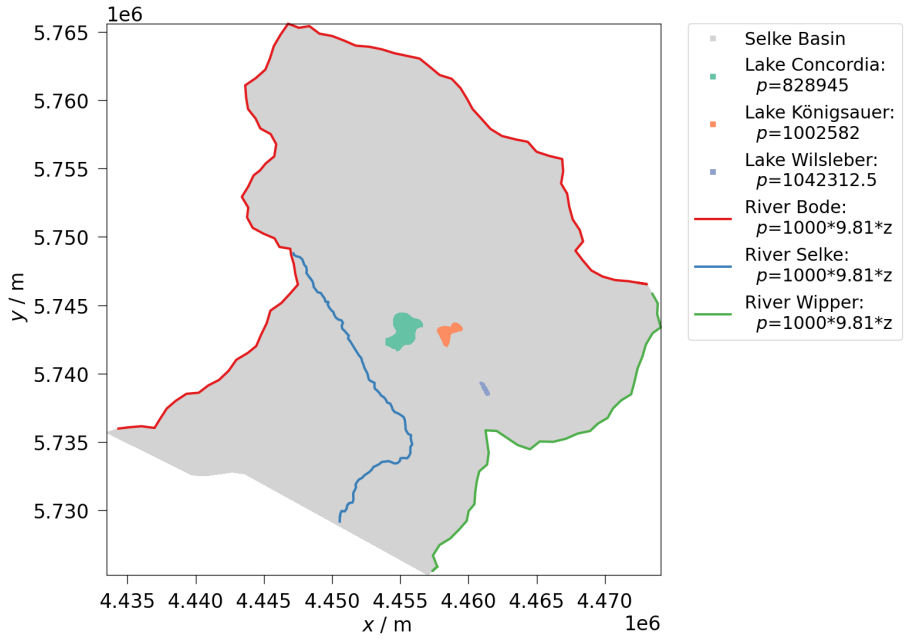


Figure 1: Model domain of the Selke basin with the Dirichlet boundary conditions at the rivers Bode, Selke and Wipper ($p = \rho_w g z$) and at the three mining lakes (fixed pressures in Pa), as plotted by `model.plot_constraints`.

3 Governing equation and model setup

The model uses the `LiquidFlow` process of OpenGeoSys [2], which describes saturated single-phase flow in porous and fractured media. For an incompressible fluid the governing equation is the volume balance of Lecture 2,

$$\beta_S \frac{\partial p_w}{\partial t} - \nabla \cdot \left(\frac{\mathbf{K}_0}{\mu_w} (\nabla p_w - \rho_w \mathbf{g}) \right) = W_0, \qquad \mathbf{q} = -\frac{\mathbf{K}_0}{\mu_w} (\nabla p_w - \rho_w \mathbf{g}), \tag{1}$$

with the storage coefficient $\beta_S = S_0/(\rho_w g)$. In a 2D horizontal model the gravity term drops out, and $h = p_w/(\rho_w g)$ plays the role of the hydraulic head; this is how the notebook converts the results.

The model is defined by an OGS project file (`Selke_Basin.prj`) that links the meshes and defines the process, media, parameters, process variables (initial, boundary and source terms) and the time loop. Instead of editing the XML by hand, the tutorial modifies it through `ot.Project`: `replace_phase_property_value`, `replace_medium_property_value`, `add_parameter`, `add_st` and `set_stepping`. The material parameters follow Nixdorf et al. [3] (Table 1).

The permeability corresponds to a hydraulic conductivity $K = K_0 \rho_w g/\mu_w \approx 5 \times 10^{-3}$ m/s, a value typical of gravel. The rivers are Dirichlet boundaries with $p = \rho_w g z$, i.e. the head is fixed at the elevation of the river; the lakes have prescribed pressures, for Lake Concordia $p = 828\,945$ Pa,

Table 1: Material parameters of the Selke base model.

Phase	Quantity	Symbol	Value	Unit
Liquid	dynamic viscosity	μ_w	1.1373×10^{-3}	Pa s
	density	ρ_w	1000	kg/m^3
Solid	intrinsic permeability	K_0	5.8×10^{-10}	m^2
	effective porosity	n_e	0.17	–
	storage	β_S	0	1/Pa
Source	groundwater recharge	W_0	$100\,\text{mm/a} \approx 3.17 \times 10^{-9}$	m/s

or 84.5 m. The recharge of 100 mm/a is applied as a volumetric source term on the whole domain. The model runs for one year with twelve monthly time steps.

4 The base case

The simulation is run with `model.run`, and the results are returned as an `ot.MeshSeries` that can be rescaled to km and days, plotted with `ot.plot.contourf` and probed at points. Figure 2 shows the final state. The head decreases from about 200 m at the southern boundary towards the Harz foreland to about 63 m along the River Bode in the north-east, and groundwater flows broadly northwards. Lake Concordia, whose level lies about 19 m below the surrounding groundwater, forms a distinct depression. The Darcy velocity is low in most of the basin but reaches its maximum of about 2.5×10^{-4} m/s around the lake, which acts as a strong sink fed from all sides. Streamlines also converge on the River Selke, which drains the aquifer along its course.

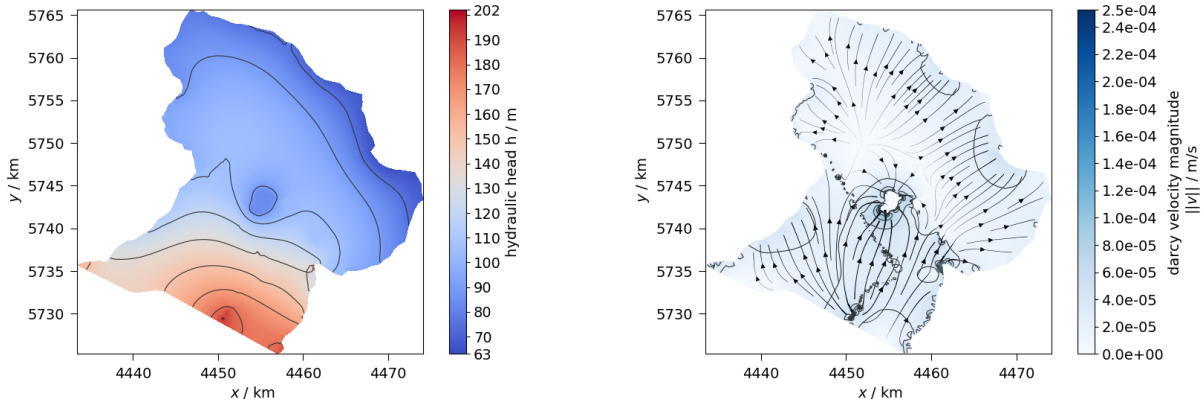


Figure 2: Base case after one year: hydraulic head with isolines (left) and Darcy velocity magnitude with streamlines on a logarithmic scale (right).

5 Scenarios

Seasonal recharge. The constant recharge is replaced by a seasonal one, scaled by $1 + \cos(2\pi t/T)$ with $T = 1$ a, so that recharge is highest in winter, zero in summer and has the same annual mean. Technically this needs only a curve (`add_curve`) and a curve-scaled parameter (`set_curve_scaled_parameter`). Figure 3 compares the heads at two observation points. Point P, close to the River Selke, hardly reacts: the Dirichlet condition at the river fixes the head nearby. Point Q, in the north far from fixed-head boundaries, follows the recharge with an amplitude of about ±16 m around 100 m.

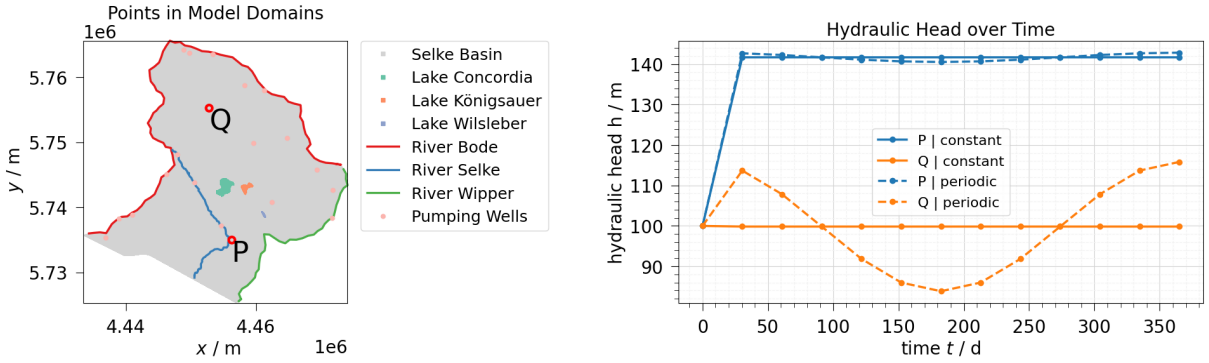


Figure 3: Observation points P and Q (left) and hydraulic head over one year for constant and seasonal recharge (right).

Raising Lake Concordia. The lake level is to rise by 18 m within the next 16 years. The scenario changes the Dirichlet value at the lake to $p_{\text{new}} = p_{\text{old}} + \Delta h\, \rho_w g = 1\,005\,525$ Pa, i.e. 102.5 m. `ot.MeshSeries.difference` gives the change in head (Fig. 4). The rise is 18 m at the lake and decays within roughly 10 km; the depression around the lake disappears, and the head gradient towards the lake, and with it the inflow, is strongly reduced. This is exactly the kind of change in hydraulic conditions that preceded the Nachterstedt landslide.

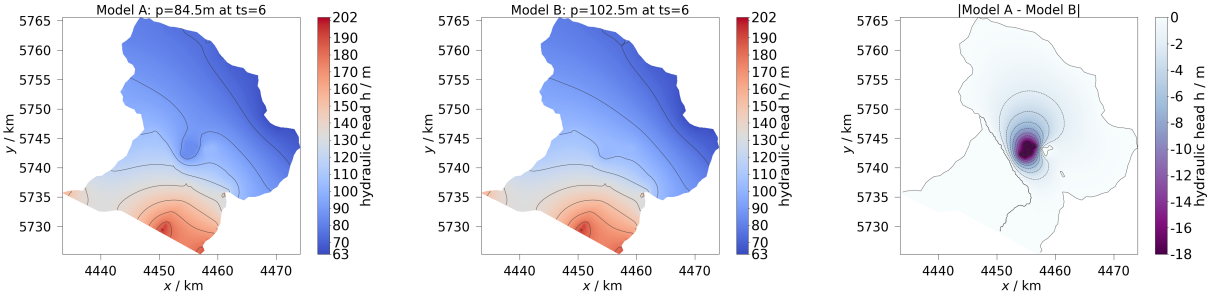


Figure 4: Hydraulic head after six months with the present (left) and raised (centre) level of Lake Concordia, and their difference (right).

6 Discussion

Read together with the lectures, the exercise makes three points.

First, the boundary conditions determine the solution. In a steady model with homogeneous properties, the heads are interpolated between the prescribed river and lake levels, modified by recharge. Point P shows no seasonal signal because it lies next to a fixed-head river, not because the aquifer there behaves differently. The archived part of the notebook therefore replaces the Dirichlet conditions at the rivers by Robin (leakage) conditions, $Q_{ex} = KWL\,(h_{\text{SW}} - h_{\text{GW}})/M$, with a bed conductance KWL/M, since fixed heads tend to overestimate the groundwater level and the river–aquifer exchange. This is the most important lesson for real-world models: the river and lake conditions are part of the conceptual model, and their type matters as much as their value.

Second, neglecting storage changes the meaning of time. With $\beta_S = 0$, Eq. (1) has no time derivative, and each monthly step is a steady state for the current recharge. The head at Q therefore follows the recharge instantly and without damping, and the first time step jumps from the initial head of 100 m to the steady state. A real aquifer, with $S_0 > 0$, would respond with a phase lag and a reduced amplitude controlled by the diffusivity K/S_0 (Lecture 2; [1]). Likewise, the 16 years for the lake rise play no role in the steady model; only the final state is computed.

Adding storage is the natural next exercise.

Third, the workflow is the main outcome. Loading meshes, editing the project file, running scenarios and computing differences all happen in Python with `ogstools`, which makes the model reproducible and the scenarios easy to vary. Participants with little OGS experience can concentrate on the hydrogeology rather than on file formats.

Some points of the notebook could be clarified in a future revision: the table of symbols gives the unit of S_0 as [1/M] instead of [1/L]; "e.g. $\partial\rho_w/\partial p_w \equiv 0$" should read "i.e."; the recharge comment gives 3.19×10^{-9} instead of 3.17×10^{-9} m/s; in the Robin formula M is used for both width and thickness, where the width should be W; the variable `p/(1000*9.81)` is the pressure head, which equals the hydraulic head only because gravity is not active in the 2D model, and this should be stated; and the section numbering jumps from 6 back to 5.2. Finally, the archived Dirichlet–Robin comparison shows the same 18 m difference around Lake Concordia as the lake scenario and no difference along the rivers, which suggests that the project copied for the Robin case already contained the raised lake level, or that the stored output is out of date. It should be re-run before it is used in teaching.

7 Conclusions

1. Exercise E3 builds a 2D groundwater model of the Selke catchment with the `LiquidFlow` process of OpenGeoSys, homogeneous parameters, Dirichlet conditions at rivers and lakes and 100 mm/a recharge.
2. The base case shows northward flow from about 200 m to 63 m head and a pronounced depression around Lake Concordia, where the Darcy velocity is highest.
3. Seasonal recharge produces head variations of about ±16 m far from rivers and almost none next to them; raising Lake Concordia by 18 m affects heads within about 10 km.
4. Because storage is set to zero, all results are sequences of steady states; transient behaviour and river–aquifer exchange (Robin conditions) are the obvious extensions.
5. For the course participants the practical message is that `ogstools` turns a real-world OGS model into a short, reproducible Python workflow in which boundary conditions and parameters can be changed and their effects compared directly.

References

[1] J. Bear (1972). *Dynamics of Fluids in Porous Media*. Environmental Science Series. American Elsevier, New York.

[2] O. Kolditz, S. Bauer, L. Bilke, N. Böttcher, J. O. Delfs, T. Fischer, U. J. Görke, T. Kalbacher, G. Kosakowski, C. I. McDermott, C. H. Park, F. Radu, K. Rink, H. Shao, H. B. Shao, F. Sun, Y. Y. Sun, A. K. Singh, J. Taron, M. Walther, W. Wang, N. Watanabe, Y. Wu, M. Xie, W. Xu, B. Zehner (2012). OpenGeoSys: an open-source initiative for numerical simulation of thermo-hydro-mechanical/chemical (THM/C) processes in porous media. *Environmental Earth Sciences* 67, 589–599. doi:10.1007/s12665-012-1546-x.

[3] E. Nixdorf, M. Sips, P. Morstein (2025). Development of a digital workflow for layer reduction strategies and visual-analytical outcome analysis to enhance geo-hydraulic modelling efficiency. *Digital Water* 3(1), 1–24. doi:10.1080/28375807.2025.2460825.

[4] Z.-Y. Zhang, C. Schmidt, E. Nixdorf, X. Kuang, J. H. Fleckenstein (2021). Effects of heterogeneous stream-groundwater exchange on the source composition of stream discharge and solute load. *Water Resources Research* 57, e2020WR029079. doi:10.1029/2020WR029079.

8 Lecture 4: Digital Twins of Geosystems

Seminar at Seoul National University

Digital Twins of Geosystems
An Essay

Olaf Kolditz
Helmholtz Centre for Environmental Research – UFZ, Leipzig

7 October 2026

Abstract

The seminar "Digital Twins of Geosystems", given by Olaf Kolditz and the OpenWorkFlow team at Seoul National University on 13 October 2026, argues that digital twins of the subsurface cannot be built directly but grow out of a mature ecosystem of workflows. The talk traces this development from the first OpenGeoSys predecessors in 1986 to OGS-6.9 in 2026. It presents workflows for data integration, model generation, uncertainty quantification and high-performance computing; the software-engineering practices and container technology that make them reproducible; and the benchmarking and process selection that make models trustworthy across scales, from the laboratory to the underground research laboratory and the repository. Only then does it define the digital twin, a virtual counterpart that describes all relevant properties and behaviours of a geosystem and provides its data through a uniform interface. This essay summarises the seminar and argues that its main message is a change of perspective: a digital twin is less a new kind of model than a sustained workflow, and its credibility rests on the software and verification infrastructure behind it.

1 Introduction

"Digital twin" has become one of the most used and least defined terms in engineering and the geosciences. For manufactured objects, an engine or a production line, the idea is clear: sensors stream data into a model that mirrors the state of the object in real time and is used to monitor, predict and control it. For geosystems, aquifers, geothermal reservoirs or radioactive waste repositories, the situation is different. The object is only partly observable, its properties are uncertain, and the relevant processes span time scales from hours to glacial cycles.

The seminar addresses this gap from the perspective of a software project with a long history (slide 1). The lineage begins with the RockFlow codes RF1–RF3 (1986–1991, Fortran and C), continues with GeoSys (2001) and OGS-5 (2008, C++), and leads to OGS-6 (2015 onwards), with the current release OGS-6.9 in 2026. Each step added capabilities: dynamic data structures, object orientation, high-performance computing, open source, software engineering, cross-platform builds, the DataExplorer, virtual reality, a Python framework, a benchmark gallery, open workflows, automation, AI support and, most recently, digital twins. The companion tools OpenWorkFlow, `ogstools` and OGSGIScape belong to the same ecosystem. The central thesis is stated at the outset: building workflows is the prerequisite for digital twins, and this requires a comprehensive ecosystem of software tools. The thesis is developed further in the opinion paper "Digital twins of geosystems" [6].

This essay follows the line of the seminar: workflows (Sect. 2), software engineering (Sect. 3), model generation by benchmarking and process selection (Sect. 4), examples across scales (Sect. 5) and finally digital twins (Sect. 6). Section 7 discusses the implications.

2 Workflows

The seminar adopts a general definition: "A workflow consists of an orchestrated and repeatable pattern of activity, enabled by the systematic organization of resources into processes that transform materials, provide services, or process information." The two key words are *repeatable* and *process information*. For geosystem analysis the information cycle has seven stages (Fig. 1): observation and monitoring, data integration, model generation, process simulation, data analytics, system optimisation and knowledge transfer, which feeds back into new observations.

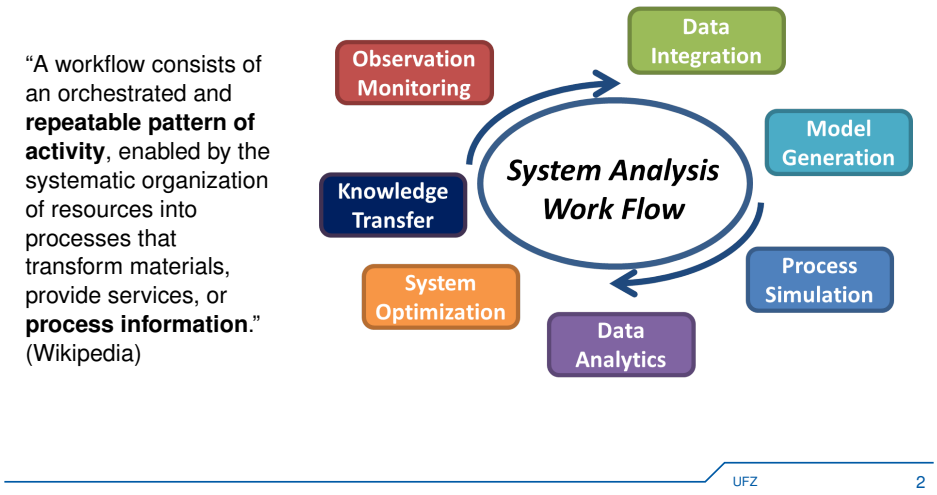


Figure 1: The system-analysis workflow as a closed cycle from observation to knowledge transfer (slide 2 of the seminar).

Four kinds of workflows illustrate the stages.

- *Data integration.* The Virtual Experiment Information System for the Mont Terri underground research laboratory combines strata, faults, laser-scanned galleries, borehole and sensor databases and simulation results in a single virtual-reality environment built with Unity and OpenGeoSys [4, 11]. An environmental information system for subsurface energy storage in Schleswig-Holstein (ANGUS project, with CAU Kiel) applies the same idea at regional scale [9].
- *Model generation.* For the CD-A and FE experiments at Mont Terri [8], geometry data, material parameters and ambient conditions are combined into a region of interest, meshed, and equipped with material groups, boundary meshes and initial conditions to yield the simulation model.
- *Uncertainty quantification.* For the FE heater experiment, a ten-step workflow based on design of experiments [2] proceeds from parameter identification, definition of history-match metrics and parameter screening through proxy building, global sensitivity analysis and Monte Carlo sampling to history-match filtering, forecast runs and the analysis of failure probabilities of safety functions.
- *High-performance computing.* Density-driven hydrothermal flow in a faulted geothermal system with 55 million nodes and 110 million unknowns [7] shows that the same model-generation chain scales to very large problems.

From these examples the seminar distils the properties a workflow must have: it must be composable from reusable parts, applicable to different applications and scales, executable in different environments, documented, standards-based, and built on quality code, quality models and a reliable software distribution.

3 Software engineering

Workflows are only repeatable if the software they use is. The seminar summarises the development practices of OpenGeoSys in four points: version control as the basis, code review, automated testing and automated delivery [1]. Together they make continuous integration possible, in which every change to the code is tested against the benchmark suite before it is released.

The second pillar is container technology. A container is a lightweight virtual machine: a virtual machine bundles an application with the files and processes of a second operating system, a container only with its files. A workflow container packs OpenGeoSys together with MFront, PHREEQC, MPI and the Python stack (NumPy, Matplotlib, PyVista, VTK). This distributes the software with all its dependencies, provides a unified runtime environment that runs in many different environments, facilitates the reuse of workflow steps and lowers the entry barrier for new users. OpenWorkFlow builds on these containers.

4 Model generation: benchmarking and process selection

A digital twin must contain the right physics, and the right physics depends on the scale. OpenGeoSys implements coupled thermal (T), hydraulic (H), mechanical (M) and chemical (C) processes. The seminar organises the choice along two axes (Fig. 2): complexity and scale. At the repository scale (REP) a thermo-hydro-mechanical (THM) model may suffice; at the underground research laboratory (URL) a THM model with Richards-type unsaturated flow (TRM) is needed; at the laboratory scale (LAB) the full non-isothermal two-phase flow in deformable media (TH2M) is resolved. Workflows connect the levels, so that parameters and insights obtained at one scale can be passed on to the next.

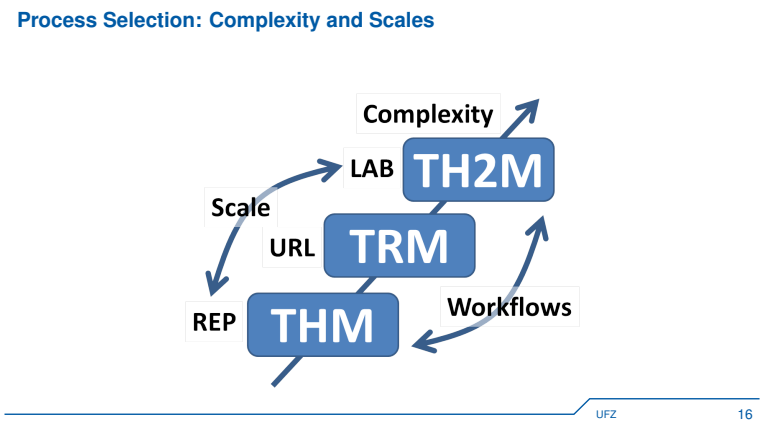


Figure 2: Process selection between complexity and scale: TH2M at laboratory, TRM at URL and THM at repository scale, linked by workflows (slide 16 of the seminar).

Trust in such models comes from benchmarking. The OGS benchmark gallery documents verified test cases for all processes. For TH2M, Grunwald et al. [5] built a systematic hierarchy in which the fully coupled formulation is reduced step by step to TH, H^2M, H^2, HM, T, H and M, and each reduced case is verified against analytical or established solutions. The Model-Hub, developed in the European projects EURAD-2 and HERMES, makes this knowledge accessible in three layers: a web layer organised by materials, processes, scales and projects; an info layer with literature and a list of benchmarks; and a JupyterLab layer with interactive notebooks.

5 Examples across scales

Three examples show the workflows at work.

- *Far field.* In the AREHS project several glaciation periods are simulated for clay and salt host rocks, a problem that covers hundreds of thousands of years and large regions.
- *Repository scale.* DECOVALEX-2019 Task E for the Bure URL proceeded in four steps: THM validation against an analytical solution [3], the TED and ALC heater experiments [10], and finally a hypothetical repository layout in Callovo-Oxfordian clay.
- *URL scale.* At Mont Terri, the CD-A, FE and FS experiments were modelled by BGR, UFZ and TUBAF and integrated with visualisation in the GeomInt and iCROSS projects.

OpenWorkFlow, the open-source synthesis platform, brings these strands together, linking near-field and far-field processes, underground research laboratories and repository models.

6 Digital twins

Only after this preparation does the seminar define its subject. Following a general definition, "a digital twin is a virtual representation that serves as the real-time digital counterpart of a physical object or process". It must

1. describe all relevant properties of the object or process,
2. show all relevant behaviours of the object or process, and
3. provide all necessary data via a uniform interface.

The components of a digital twin of a geosystem (Fig. 3) are geology, building information models (BIM), geographic information systems (GIS), material data, operational data, sensor data and models, joined by an interface. Possible purposes are on-the-fly updating of model parameters, monitoring of the repository and continuous model validation. Further properties are a distributed, multi-agent implementation, versioning, composability, in which more complex twins are built from a common core, integration with workflows, and the need for a robust basis.

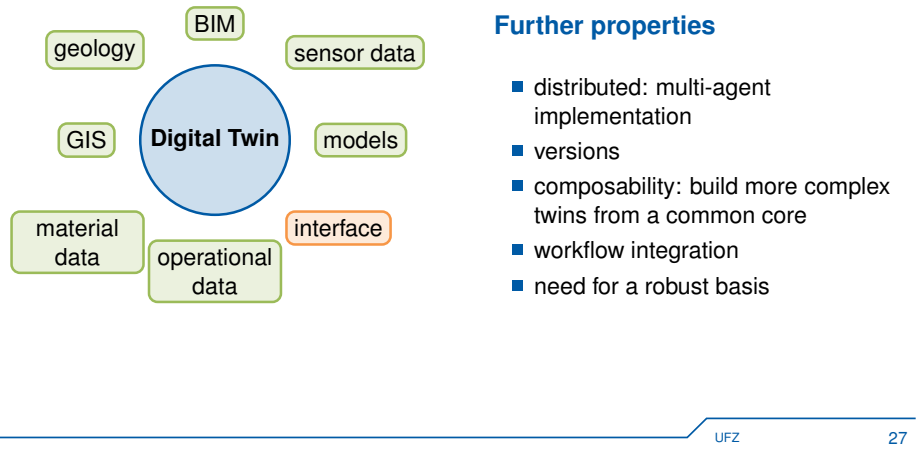


Figure 3: Components of a digital twin of a geosystem and its further properties (slide 27 of the seminar).

Two demonstrators close the talk. GeoDT shows a twin of the GeoLaB geothermal research laboratory with a fault zone, galleries and boreholes. The campus twin of the SolutionLab Urban-LE at the Wissenschaftspark Leipzig links buildings with their energy, water and heating data, and shows that the same concept extends from the subsurface to the urban environment above it.

7 Discussion

Read against the rest of the course, the seminar makes three points.

First, a digital twin is a workflow, not a model. Its defining feature is the closed loop of Fig. 1: data update the model, the model predicts, new data confirm or correct the prediction. Every element of that loop already exists in the course material, from mesh generation and project files to simulation and analysis with `ogstools` in the Selke exercise. What the twin adds is that the loop runs continuously and automatically. This explains the order of the talk: without repeatable, containerised and tested workflows there is nothing that could run continuously.

Second, "real time" means something different for geosystems. A repository, an aquifer or a geothermal reservoir evolves over years to millennia, and sensors sample only a tiny fraction of its volume. A twin of a geosystem is therefore model-centric rather than sensor-centric: the model fills in what cannot be observed, and its credibility depends on verification (benchmarks), validation (experiments such as Mont Terri) and quantified uncertainty (history matching). The uncertainty-quantification workflow of Sect. 2 is thus not an add-on but the core of continuous model validation, one of the purposes named for the twin.

Third, process selection is part of the twin. A twin that "shows all relevant behaviours" must know which behaviours are relevant at which scale. The complexity–scale diagram of Fig. 2 and the TH2M benchmark hierarchy provide this knowledge in a form that can be reused. The groundwater flow equation of Lecture 2 is the bottom of this hierarchy; the TH2M formulation is its top.

Some points of the slides could be clarified in a future revision: the definition on slide 25 calls for a real-time counterpart, which the later discussion of geosystems implicitly relaxes, and this could be said explicitly; the purposes on slide 26 name "monitoring of the repository" although the twin is introduced for geosystems in general; slide 22 cites both [Wang2021] and [Wan+21], of which only the latter appears in the reference list; [Gra+22] is still listed as "submitted"; the reference list does not yet include the opinion paper itself; and slides 13 and 14 are duplicates except for the OpenWorkFlow overlay.

8 Conclusions

1. The seminar presents digital twins of geosystems as the outcome of four decades of code development, from RockFlow (1986) to OGS-6.9 (2026), and of more than five years of workflow development.
2. Workflows for data integration, model generation, uncertainty quantification and HPC are the building blocks of a twin; they must be composable, reusable, documented and portable.
3. Software-engineering practices (version control, code review, automated tests and delivery) and containers make these workflows reproducible.
4. Benchmarking and scale-dependent process selection, from TH2M in the laboratory to THM at repository scale, make the models behind the twin trustworthy.
5. A digital twin describes the relevant properties and behaviours of a geosystem and provides its data through a uniform interface; for the subsurface its credibility rests on verification, validation and uncertainty quantification rather than on dense real-time sensing.

References

[1] L. Bilke, B. Flemisch, T. Kalbacher, O. Kolditz, R. Helmig, T. Nagel (2019). Development of open-source porous media simulators: principles and experiences. *Transport in Porous Media* 130(1), 337–361. doi:10.1007/s11242-019-01310-1.

[2] J. Buchwald, A. A. Chaudhry, K. Yoshioka, O. Kolditz, S. Attinger, T. Nagel (2020). DoE-based history matching for probabilistic uncertainty quantification of thermo-hydro-mechanical processes around heat sources in clay rocks. *International Journal of Rock Mechanics and Mining Sciences* 134, 104481. doi:10.1016/j.ijrmms.2020.104481.

[3] A. A. Chaudhry, J. Buchwald, O. Kolditz, T. Nagel (2019). Consolidation around a point heat source (correction and verification). *International Journal for Numerical and Analytical Methods in Geomechanics* 43(18), 2743–2751. doi:10.1002/nag.2998.

[4] N. Graebling, Ö. O. Şen, L. Bilke, T. Cajuhi, D. Naumov, W. Wang, G. Ziefle, D. Jaeggi, J. Maßmann, G. Scheuermann, O. Kolditz, K. Rink (2022). Prototype of a virtual experiment information system for the Mont Terri underground research laboratory. *Frontiers in Earth Science*.

[5] N. Grunwald, J. Maßmann, D. Naumov, O. Kolditz, T. Nagel (2022). Non-isothermal two-phase flow in deformable porous media: systematic open-source implementation and verification procedure. *Geomechanics and Geophysics for Geo-Energy and Geo-Resources* 8. doi:10.1007/s40948-022-00394-2.

[6] O. Kolditz, L. Bilke, T. Cajuhi, M. Baes, N. Bartels, C. Budach, et al. Digital twins of geosystems. Opinion paper, *Environmental, Geological and Geotechnical Engineering* (Elsevier).

[7] V. I. Malkovsky, F. Magri (2016). Thermal convection of temperature-dependent viscous fluids within three-dimensional faulted geothermal systems: estimation from linear and numerical analyses. *Water Resources Research* 52(4), 2855–2867. doi:10.1002/2015WR018001.

[8] H. R. Müller, B. Garitte, T. Vogt, S. Köhler, T. Sakaki, H. Weber, T. Spillmann, M. Hertrich, J. K. Becker, N. Giroud, V. Cloet, N. Diomidis, T. Vietor (2017). Implementation of the full-scale emplacement (FE) experiment at the Mont Terri rock laboratory. *Swiss Journal of Geosciences* 110(1), 287–306. doi:10.1007/s00015-016-0251-2.

[9] K. Rink, Ö. O. Şen, M. Schwanebeck, T. Hartmann, F. Gasanzade, J. Nordbeck, S. Bauer, O. Kolditz (2022). An environmental information system for the exploration of energy systems. *Geothermal Energy* 10. doi:10.1186/s40517-022-00215-5.

[10] W. Wang, H. Shao, T. Nagel, O. Kolditz (2021). Analysis of coupled thermal-hydro-mechanical processes during small scale in situ heater experiment in Callovo-Oxfordian clay rock introducing a failure-index permeability model. *International Journal of Rock Mechanics and Mining Sciences* 142, 104683. doi:10.1016/j.ijrmms.2021.104683.

[11] G. Ziefle, T. Cajuhi, N. Graebling, D. Jaeggi, O. Kolditz, H. Kunz, J. Maßmann, K. Rink (2022). Multi-disciplinary investigation of the hydraulic-mechanically driven convergence behaviour: CD-A twin niches in the Mont Terri Rock Laboratory during the first year. *Geomechanics for Energy and the Environment*. doi:10.1016/j.gete.2022.100325.

Additional course material

Repositories:
GitHub: https://github.com/OlafKolditz/ogs-course-2026